

A Complementary Approach to Incorrectness Typing

CELIA MENGYUE LI, University of Bristol, United Kingdom

SOPHIE PULL, University of Bristol, United Kingdom

STEVEN RAMSAY, University of Bristol, United Kingdom

We introduce a new two-sided type system for verifying the correctness and incorrectness of functional programs with atoms and pattern matching. A key idea in the work is that types should range over sets of normal forms, rather than sets of values, and this allows us to define a complement operator on types that acts as a negation on typing formulas. We show that the complement allows us to derive a wide range of refutation principles within the system, including the type-theoretic analogue of co-implication, and we use them to certify that a number of Erlang-like programs go wrong. An expressive axiomatisation of the complement operator via subtyping is shown decidable, and the type system as a whole is shown to be not only sound, but also complete for normal forms.

CCS Concepts: • **Theory of computation** → *Logic and verification*; **Type theory**; **Program reasoning**.

Additional Key Words and Phrases: type systems, refutation, higher-order program verification, incorrectness

ACM Reference Format:

Celia Mengyue Li, Sophie Pull, and Steven Ramsay. 2026. A Complementary Approach to Incorrectness Typing. *Proc. ACM Program. Lang.* 10, POPL, Article 82 (January 2026), 29 pages. <https://doi.org/10.1145/3776724>

1 Introduction

Since at least Milner’s influential 1978 paper [19], an important role of type systems in programming languages has been to provide a formal mechanism for reasoning statically about how programs will behave at runtime. In this role, types A are understood as a syntax for describing properties of (the behaviour of) program expressions M , and a derivation of the typing $M : A$ amounts to a proof that M satisfies A .

It is often said that a typical purpose of such reasoning systems is the detection of runtime errors [11, 25]. However, in a traditional type system, the means by which this benefit is realised is somewhat indirect, because the type system is designed only to certify the absence of such errors. When a program is not well typed, it may be that it harbours a runtime error, but it could also be that the program simply lies beyond reasoning power of the type system.

To be able to detect runtime errors, we need a system that can reason about programs that go wrong. However, such a capacity is incompatible with the prevailing approach to type systems for programming languages. Typically, the soundness of the type system implies that *well-typed programs cannot go wrong*, and since it is also typical that we can only construct type derivations for well-typed programs, it follows that we cannot derive properties of a program that goes wrong. Such programs simply don’t exist as far as the type system is concerned.

Authors’ Contact Information: Celia Mengyue Li, University of Bristol, Bristol, United Kingdom, celia.li@bristol.ac.uk; Sophie Pull, University of Bristol, Bristol, United Kingdom, sophie.pull@bristol.ac.uk; Steven Ramsay, University of Bristol, Bristol, United Kingdom, steven.ramsay@bristol.ac.uk.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/1-ART82

<https://doi.org/10.1145/3776724>

An alternative approach was proposed by Ramsay and Walpole in their POPL'24 paper [28]. Their *two-sided type systems* allow for an extended form of typing judgement, whose shape is generally as follows:

$$M_1 : A_1, \dots, M_k : A_k \vdash N_1 : B_1, \dots, N_m : B_m$$

Two-sided type systems lift the restriction that typing formulas $M : A$ on the left of the turnstile must be variable typings (i.e. M must be a variable), and the restriction that there must be exactly one typing formula on the right-hand side. However, the meaning of such a judgement smoothly generalises that of the traditional typing judgement: if *all* of the M_i reduce to values of type A_i , then *some* N_i is sure to reduce to a value of type B_i (or diverge in the process).

In addition to the usual kinds of typing judgements, which appear as a special case, two-sided judgements can express that certain conditions are *necessary* for proper evaluation. For example, consider a typical definition of exponentiation x^y as a fixed point involving multiplication, subtraction and an equality test (defined only) on integers¹: $\text{power} := \text{fix } f. \lambda xy. \text{if } y = \underline{0} \text{ then } \underline{1} \text{ else } x * (f \ x \ (y - \underline{1}))$. In the system of this paper, it is both true and provable that:

$$\text{power } x \ y : \text{Int} \vdash y : \text{Int}$$

In other words, *if* the term $\text{power } x \ y$ reduces to a value, *then* the second input, y , *must* have been an integer. By contrast, however, the following judgement, is *not* true:

$$\text{power } x \ y : \text{Int} \vdash x : \text{Int}$$

It states that, for the power function to produce an integer value, the first input must have been an integer: *if* $\text{power } x \ y$ evaluates to a integer, *then* x is an integer. A quick perusal of the definition of power , however, suffices to convince oneself that it is not *necessary*; the term $\text{power } (\lambda z. z) \ \underline{0}$ will evaluate to a integer value without complaint.

Two-sided type systems sidestep the problem of $\vdash M : A$ being derivable only for M that cannot go wrong by allowing to prove a kind of negation $M : A \vdash$. The meaning of this judgement is: if M evaluates to a value of type A then false, or simply M cannot evaluate to a value of type A . With a type Ok representing all values, one can prove, for example:

$$\text{power } \underline{0} \ (\lambda z. z) : \text{Ok} \vdash$$

Since Ok is the type of *all* values, a proof of this judgement is a certificate that exponentiation of zero and the identity function, *in that order*, either diverges or is sure to go wrong.

However, the original development of two-sided type systems in [28] had some significant weaknesses, which we shall summarise here, and discuss in detail in the main body of the paper. First, although the form of judgment is as above, most of the metatheory was developed only under the *single-subject restriction* in which at most one of the $M_1, \dots, M_k, N_1, \dots, N_m$ is not a variable. Second, although the two-sided setup naturally lends itself to a notion of complement on types, it was accommodated at the expense of a weaker soundness guarantee. Third, the systems of that work are formed from a large number of primitive rules, some overlapping, and with many not having a clear justification for their inclusion in the theory.

1.1 A Two-Sided System with Complement on Normal Forms

In this paper, we present a new two-sided type system which both overcomes these weaknesses and is an interesting type system in its own right. As well as certifying that programs cannot go wrong (as usual), the system can also prove that programs cannot go right. This latter capacity gives it a similar power to O'Hearn's Incorrectness Logic for first-order imperative programs [21], and Lindahl and Sagonas' Success Types for Erlang [17] (though neither of these have the former

¹In this paper we write the integer literal corresponding to n as $\underline{n}$

capacity). Inspired by this connection, we use our system to certify the defectiveness of a number of textbook Erlang examples due to [13], including the following program, which is a textbook example used to demonstrate the limitations of Dialyzer's type inference, and which our system can prove to be incorrect:

```

let money = λxy. ('give, (x, y)) in
let item = λx. match x with
    | ('count, ('give, (y, z)))   ↦ y;
    | ('account, ('give, (y, z))) ↦ z; in
let op = λxy. x + y in
let tup = money 5 'you in
    op (item ('count, tup)) (item ('account, tup))

```

The reasoning encoded in the typing derivation is roughly as follows. The expression in the innermost body of the let, $op(item('count, tup))(item, ('account, tup))$ fails to reduce to a value whenever *all* of the following are true:

- (i) op is a function that requires that its second argument evaluates to an integer
- (ii) and, $item$ is a function that, to return an integer, requires an input that *does not* reduce to a value of shape $('account, ('give, (P, Q)))$ where Q is not an integer
- (iii) and, tup *does* reduce to a value of shape $('give, (P, Q))$ where Q is not an integer.

Since a typing can be derived for each of op , $item$ and tup certifying that they *do* satisfy (i), (ii) and (iii), we must conclude that the innermost body of the let, and therefore the program as a whole does not reduce to a value.

A key new idea in our work is that types should not be limited to describing sets of values, but instead should describe sets of normal forms more generally. To motivate this idea, let us make a little more precise what is meant by *going wrong*. In general, each type system has its own notion of what it means to go wrong, because type systems have been designed to guarantee various different notions of safety, such as the classical type safety of Milner, memory safety, thread safety and so on. However, it seems reasonable to consider safety properties in general, that is, that programs do not reach some kind of *bad state*.

An elegant way to define bad states in a functional programming languages is as those program expressions that are neither values nor that can make a reduction step: we will call them *stuck terms*. In the language we study, we will consider both a classical notion of type safety, so that e.g. applying a number to a function, $\underline{3}(\lambda x. x)$, is a stuck term, and we will consider pattern-matching safety, so that, e.g. failing to match every branch in a case analysis expression, $match('a, \underline{3})$ with $\{('b, x) \mapsto x\}$, is also stuck.

In the traditional approach to safety via type systems, one would establish a soundness theorem to ensure that any typable program $M : A$ cannot reduce to a stuck term. In other words, if typable program M normalises then its normal form must be a value of the type A . This is therefore consistent with the usual semantic interpretation of A as a certain set of values (modulo divergence). Since, for incorrectness, we want to reason about programs that go wrong, it follows that we should allow types that number among their inhabitants also the stuck terms. Since stuck terms and values, together, constitute all possible normal forms, we see the type system as a formal system for reasoning about how terms will reduce to normal form, in general, and types simply as sets of normal forms.

The interpretation of types as sets of normal forms allows us to axiomatise a complement operator on types, the type A^c is interpreted as all normal forms that are not in the type A , *without* weakening our soundness guarantees. Moreover, alongside a reasonably expressive subtyping

relation, it allows us to form the type system from a small number of well-motivated rules, and yet induce an extremely rich theory – we show that a wide variety of interesting typing principles are derivable, including analogues of all the key refutational rules of [28].

In fact, complement with respect to normal forms arises naturally in any consideration of typing for incorrectness, and not only in the two-sided setting. If we want to be able to assign types to terms that go wrong, then we must be able to describe, within the type system, the conditions under which terms of different shapes should be considered stuck. For example, an integer addition $M + N$ is stuck if M is stuck, or if M is some value other than an integer. In other words, $M + N$ is stuck whenever M is some *normal form* that is *not* in the type Int of integer values, that is, when $M : \text{Int}^c$ is derivable. Similarly, the second projection $\pi_2(M)$ of some term M is stuck whenever M is either itself stuck or when M is some value other than a pair value. In other words, when M is some *normal form* that is *not* in the type of all pair values, or $M : \text{PairVal}^c$ for short. Function applications $M N$ are stuck whenever M is a normal form *not* in the type of all functions, and so on. In fact, we are able to show that not only is our type system sound, but it admits a complete classification of normal forms: every normal form can be proven either to be a value or to be a stuck term. By contrast, most type systems in the literature do not satisfy even the value part of this property, that is, it is typical that there are values that are not well typed.

Contributions. In sum, our contributions are as follows:

- (i) We define a two-sided type system for a functional programming language with atoms and pattern matching. The system exploits two-sidedness to give a natural and expressive rule for typing pattern-matching. Unlike [28], the system is formed from a small number of well-motivated rules.
- (ii) We show that the refutational power of the system is, nevertheless, excellent, by deriving (analogues of) the key refutation typing rules from the original work (Theorem 4.1). Moreover, we show the potential of the idea by certifying that a sequence of textbook Erlang programs due to Hébert [13] all go wrong.
- (iii) We introduce the coarrow type $A \multimap B$ as a type-theoretic counterpart to coimplication (as featured in co- and bi-intuitionistic logic [9, 29, 31]), and show that it is the appropriate conceptual tool with which to reason refutationally about functions.
- (iv) We show soundness of the subtype relation via a natural set-theoretic model (Theorem 6.1), and we show its decidability by describing an equivalent alternative characterisation (Theorem 6.3), whose decision problem can be reduced to bottom-up proof search (Theorem 6.4).
- (v) We show that the system is equivalent to a one-sided system with multiple conclusions (Theorem 7.2), that is, in every judgement $\Gamma \vdash \Delta$, the assumptions Γ may be taken to comprise only typings of shape $x : A$. By introducing a new technical device for working with non-single-subject Δ , we argue progress and preservation for the one-sided system, and thus obtain soundness of the original system (Theorem 7.4).

Outline. The rest of the paper is structured as follows. We introduce the programming language and its semantics in Section 2; the types, subtyping relation and type system in Section 3. We then derive the key principles of refutation and put them to work certifying Hébert’s Erlang examples in Section 4. This leads into a discussion of how to express necessity in conjunction with curried functions and the introduction of the coarrow as counterpart to coimplication in Section 5. We then develop the metatheory of subtyping in Section 6 and of the type system in Section 7. We conclude with a discussion of related work in Section 8. A long version of the paper containing all the proofs is [16].

2 A Functional Programming Language

We define a functional language with integers, pairing, recursive functions, Erlang-style atoms and matching.

Definition 2.1 (Terms and Patterns). Assume a denumerable set of term variables x, y, z and a denumerable set of atoms 'a, 'b, 'c including the distinguished atoms 'true and 'false. The patterns of the language, typically p, q are:

$$p, q ::= x \mid \text{'a} \mid (p, q)$$

Patterns are required to be *linear* in the sense that each variable occurs in p at most once.

The *terms* of the language, typically M, N, P, Q , are defined by the following grammar. Here, $\underline{n}$ denotes the numeral for integer n , the symbol $\otimes$ represents any reasonable choice of binary operations on integers, e.g. $+$, $-$, $*$, and $\bowtie$ any reasonable choice of binary relations on integers, e.g. $\leq$, $=$ and so on.

$$M, N ::= x \mid \text{'a} \mid \underline{n} \mid (M, N) \mid \lambda x. M \mid MN \mid \text{fix } x. M \mid M \otimes N \mid M \bowtie N \mid \text{match } M \text{ with } \{\{_{i=1}^k p_i \mapsto N_i\}$$

The patterns $p_1, \dots, p_k$ in a match expression are required to be *disjoint*, that is, a given term will match at most one. We consider terms identified up to renaming of bound variables and adopt the usual conventions regarding their treatment. The set of free variables of a term M is written $\text{FV}(M)$. A term with no free variables is said to be *closed*.

A *term substitution*, typically σ, τ , is a finite map from term variables to terms. We write the application of a substitution σ to a term M postfix, as in $M\sigma$. We use $\text{dom}(\sigma)$ for the domain of the map σ . We write concrete substitutions explicitly as a list of maplets, as in $[M_1/x_1, \dots, M_n/x_n]$.

Finally, the *values*, typically V, W , are those *closed* terms that can be described by the following:

$$V, W ::= \text{'a} \mid \underline{n} \mid (V, W) \mid \lambda x. M$$

Most of the language is standard, we mention only that: atoms 'a are literals without internal structure, they may only be compared via pattern matching, and the pattern match expression $\text{match } M \text{ with } \{\{_{i=1}^k p_i \mapsto N_i\}$ scrutinises the term M , and if M matches one of the (disjoint) patterns p_i then it will evaluate as N_i .

Abbreviations. For $i \in [1, 2]$ we write $\pi_i(M)$ as an abbreviation for $\text{match } M \text{ with } \{(x_1, x_2) \mapsto x_i\}$. We write div for $\text{fix } x. x$. We write if M then N else P as an abbreviation for $\text{match } M \text{ with } \{\text{'true} \mapsto N \mid \text{'false} \mapsto P\}$.

The language reduces according to a call-by-name strategy². However, in the interests of keeping the semantics of pattern matching simple, and a little closer to that of Erlang, we require that pairing and matching are strict. We reduce the left-component of a pair first, and only if it reduces to a value do we reduce the second component.

Definition 2.2 (Reduction). The *evaluation contexts* are defined by the following grammar:

$$\mathcal{E} ::= [\cdot] \mid \mathcal{E} N \mid (\mathcal{E}, N) \mid (V, \mathcal{E}) \mid \mathcal{E} \otimes N \mid \underline{n} \otimes \mathcal{E} \mid \mathcal{E} \bowtie N \mid \underline{n} \bowtie \mathcal{E} \mid \text{match } \mathcal{E} \text{ with } \{\{_{i=1}^k p_i \mapsto N_i\}$$

Given a context $\mathcal{E}$, we write $\mathcal{E}[M]$ for the term obtained by replacing the hole $[\cdot]$ by M . The *one-step reduction relation*, $M \triangleright N$, is the binary relation on terms obtained as the closure of the following

²This matters little for the development, but we were interested to explore CBN as [28] anticipates some difficulty when changing the meaning of typing formulas on the left of the turnstile to accommodate the fact that they may not evaluate. However, we observed that the system is already sound for call-by-name without any change to the semantics, the crux of which is contained in Remark 3.2

schema under evaluation contexts. The terms on the left-hand side of the schema are said to be *redexes*. We write $\triangleright^*$ for the reflexive transitive closure.

$$\begin{array}{ll}
 (\lambda x. M) N \triangleright M[N/x] & \text{fix } x. M \triangleright M[\text{fix } x. M/x] \\
 \pi_i(V_1, V_2) \triangleright V_i \quad (i \in \{1, 2\}) & \underline{n} \otimes \underline{m} \triangleright \underline{n} \otimes \underline{m} \\
 \text{match } V \text{ with } \{\downarrow_{i=1}^k p_i \mapsto N_i\} \triangleright N_j \sigma \quad (V = p_j \sigma) & \underline{n} \bowtie \underline{m} \triangleright '(n \bowtie m)
 \end{array}$$

Definition 2.3 (Normal forms, values, stuck terms and going wrong). A term that cannot make a step is said to be in *normal form*, and we reserve the letter U as a metavariable for those terms that are in normal form. *Closed* normal forms can be partitioned into two disjoint sets: the values, which we have already introduced, and the stuck terms. A closed term is said to be a *stuck term*, for which we use the metavariables S and T , just if it is a normal form but *not* a value. A term is said to *go wrong* just if it reduces to a stuck term, and to *evaluate* just if it reduces to a value.

LEMMA 2.4 (STUCK TERMS). *The stuck terms may equivalently be described as those closed terms that can be described by the following grammar:*

$$\begin{array}{l}
 S, T ::= (S, M) \mid (V, S) \\
 \quad \mid U \otimes M \mid \underline{n} \otimes U \mid U \bowtie M \mid \underline{n} \bowtie U \quad \text{when } U \text{ is not of shape } \underline{n} \\
 \quad \mid U M \quad \text{when } U \text{ is not of shape } \lambda x. M \\
 \quad \mid \text{match } U \text{ with } \{\downarrow_{i=1}^k p_i \mapsto N_i\} \quad \text{when } U \text{ is a value and } \forall i. \forall \sigma. U \neq p_i \sigma, \text{ or } U \text{ stuck}
 \end{array}$$

Observe the use of negation in the above definition: it is this that will later make the complement operator on types a particularly useful conceptual tool in reasoning about terms that go wrong. For example, we will be able to encode the side condition on stuck terms of shape $U M$ by the typing formula $U : \text{Fun}^c$. Note also that, in stuck terms of shape (S, M) , the term M in the second component is arbitrary – it may not even be a normal form. For example, $(\underline{1}(\lambda x. x), \underline{\text{div}})$ is stuck, because pairs are evaluated eagerly from left-to-right, reduction stops once the first component becomes stuck.

3 Two-Sided System

We start by defining the language of types and associated pattern type substitutions.

Definition 3.1 (Types). The types of the system, typically A, B, C , are given by the following:

$$A, B ::= \text{Top} \mid \text{Ok} \mid \text{Atom} \mid \text{Int} \mid 'a \mid (A, B) \mid A \rightarrow B \mid \bigvee_{i=1}^k A_i \mid A^c$$

We define a *pattern type substitution* as a finite mapping, typically θ , from *term* variables to types. Like term substitutions, we will often write pattern type substitutions concretely using the notation $[A_1/x_1, \dots, A_n/x_n]$, and apply them postfix.

The types can be assigned meanings as sets of closed normal forms. Making precise this meaning is useful for intuition, and it will be used formally in the statement and proof of the soundness of subtyping, Theorem 6.1. For each type A , we define the set of closed normal forms $\llbracket A \rrbracket$ and the set of closed terms $\llbracket A \rrbracket_*$, by mutual induction on A :

$$\begin{array}{ll}
 \llbracket A \rrbracket_* &= \{ M \mid \text{if } M \text{ reduces to a normal form } U, \text{ then } U \in \llbracket A \rrbracket \} \\
 \llbracket \text{Top} \rrbracket &= \{ U \mid U \text{ is a closed normal form} \} & \llbracket \text{Int} \rrbracket &= \{ \underline{n} \mid n \in \mathbb{Z} \} \\
 \llbracket \text{Ok} \rrbracket &= \{ V \mid V \text{ is a value} \} & \llbracket \bigvee_{i=1}^k A_i \rrbracket &= \bigcup_{i=1}^k \llbracket A_i \rrbracket \\
 \llbracket \text{Atom} \rrbracket &= \{ 'a \mid 'a \text{ is an atom} \} & \llbracket A^c \rrbracket &= \llbracket \text{Top} \rrbracket \setminus \llbracket A \rrbracket \\
 \llbracket A \rightarrow B \rrbracket &= \{ \lambda x. M \mid \forall N \in \llbracket A \rrbracket_*. M[N/x] \in \llbracket B \rrbracket_* \} & \llbracket 'a \rrbracket &= \{ 'a \} \\
 \llbracket (A, B) \rrbracket &= \{ (V, U) \mid V \in \llbracket A \rrbracket \cap \llbracket \text{Ok} \rrbracket, U \in \llbracket B \rrbracket \} \cup \{ (S, N) \mid S \in \llbracket A \rrbracket \cap \llbracket \text{Ok}^c \rrbracket, N \in \llbracket B \rrbracket_* \}
 \end{array}$$

As can be seen from the semantics of types, the idea is that each type represents a certain set of normal forms. Top is the type of all normal forms, Ok is the type of all values, Atom the type of all atoms and Int the type of all integers.

We use the syntax (A, B) for the type of pairs whose components are of type A and B respectively, rather than, say $A \times B$. This is convenient for our later discussion of the typing of pattern matching, because it means that we can discuss a term instance $p\sigma$ of a pattern p , and its type $p\theta$ using the same notation and a *pattern type substitution*. For example, we will be able to prove that $\vdash (x, y)[(\lambda z. z)/x, _ / y] : (x, y)[(\text{Int} \rightarrow \text{Int})/x, \text{Int}/y]$. Note that the type (A, B) does not necessarily contain only values – it can be that an element of this type is a stuck term whenever either A or B may contain a stuck term. In particular, one can see from the definition that the right summand contains all stuck terms of shape (S, N) where S is itself stuck and N is arbitrary (may not even be a normal form). Also, it may be that (A, B) is empty in the case that A is empty (e.g. when $A = \text{Top}^c$).

As usual, the type $A \rightarrow B$ represents those functions that guarantee to map a given term of type A into a normal form of type B , or diverge in the process. Note that we use the term form of the semantics $\llbracket A \rrbracket_*$ for both argument and result since our calling convention is call-by-name (but see also Remark 3.2). Note that the type $A \rightarrow B$ is *never* empty, since it always contains $\lambda x. \text{div}$ – a function that inevitably diverges certainly satisfies the condition on membership.

The semantics of types justifies the following further abbreviations:

$$\begin{array}{llll} \text{Bot} := \text{Top}^c & \text{PairVal} := (\text{Ok}, \text{Ok}) & A \multimap B := A^c \rightarrow B^c & \text{Bool} := \text{'true'} \vee \text{'false'} \\ \text{Pair} := (\text{Top}, \text{Top}) & \text{Fun} := \text{Bot} \rightarrow \text{Top} & \bigwedge_{i=1}^k A_i := (\bigvee_{i=1}^k A_i)^c & \end{array}$$

Notice that Pair is the type of all normal forms constructed using pairing, and thus includes those pairings that describe stuck terms. By contrast, PairVal is the type of all pair values (pairs whose components are again values). Note also that we are able to construct a type whose interpretation is exactly the set of all functions, Fun , simply as an abbreviation. This would be difficult if our types were restricted to sets of values only, since a function $\lambda x. M$, whose M immediately goes wrong, would not live inside any type of shape $A \rightarrow B$.

Remark 3.2. The type $A \multimap B$, pronounced *A only to B*, which was introduced in [28], is defined simply as an abbreviation for $A^c \rightarrow B^c$. This makes sense because the defining condition for $A \multimap B$ is “if $M[N/x]$ reduces to a normal form in $\llbracket B \rrbracket_*$ then $N \in \llbracket A \rrbracket_*$,” and this is the contrapositive of the condition “if $N \notin \llbracket A \rrbracket_*$ then $M[N/x] \in \llbracket B^c \rrbracket_*$.” Unfolding the definitions, this latter condition is equivalent to “if N reduces to a normal form in $\llbracket A^c \rrbracket_*$ then $M[N/x] \in \llbracket B^c \rrbracket_*$.” Finally, note that, a basic property of reduction in our language is that any abstraction $\lambda x. M$ will satisfy this condition iff it satisfies the condition “if $N \in \llbracket A^c \rrbracket_*$ then $M[N/x] \in \llbracket B^c \rrbracket_*$.” To see this, suppose $\lambda x. M$, on input N that reduces to an A^c normal form, either diverges or returns a B^c normal form. Then $\lambda x. M$, on input N which *either diverges or* reduces to an A^c normal form, must still in the end either diverge or return a B^c normal form.

3.1 Subtype Relation

A key component of the type system is a subtype relation, $A \sqsubseteq B$, which axiomatises some key properties of complement. Later, in Theorem 6.1, we make precise the idea that the meaning of such an assertion is $\llbracket A \rrbracket \subseteq \llbracket B \rrbracket$.

Definition 3.3 (Subtyping Relations). The subtyping relation is defined inductively by the rules in Figure 1. The relation $A \parallel B$ holds just if $A \neq B$ and, *either* A and B are both atoms *or* $A, B \in \{\text{Int}, \text{Pair}, \text{Fun}, \text{Atom}\}$. Define *subtype equivalence*, written $A \sim B$, just if $A \sqsubseteq B$ and $B \sqsubseteq A$. Since the

REFL	TOP	OK	DISJ	ATOM
$\frac{}{A \sqsubseteq A}$	$\frac{}{A \sqsubseteq \text{Top}}$	$\frac{}{\text{Int} \vee \text{PairVal} \vee \text{Fun} \vee \text{Atom} \sqsubseteq \text{Ok}}$	$\frac{}{A \sqsubseteq B^c} A \parallel B$	$\frac{}{a \sqsubseteq \text{Atom}}$
PAIRC	PAIR		FUN	
$\frac{}{(A^c, \text{Top}) \vee (\text{Ok}, B^c) \sqsubseteq (A, B)^c}$	$\frac{A \sqsubseteq A' \quad B \sqsubseteq B'}{(A, B) \sqsubseteq (A', B')}$	$\frac{A' \sqsubseteq A \quad B \sqsubseteq B'}{A \rightarrow B \sqsubseteq A' \rightarrow B'}$		
UNIONL	UNIONR	COMPL	COMPR	TRANS
$\frac{B_i \sqsubseteq A \ (\forall i \in [1, k])}{\bigvee_{i=1}^k B_i \sqsubseteq A}$	$\frac{A \sqsubseteq B_j}{A \sqsubseteq \bigvee_{i=1}^k B_i} \ j \in [1, k]$	$\frac{B^c \sqsubseteq A}{A^c \sqsubseteq B}$	$\frac{B \sqsubseteq A^c}{A \sqsubseteq B^c}$	$\frac{A \sqsubseteq C \quad C \sqsubseteq B}{A \sqsubseteq B}$

Fig. 1. Subtype relation on types

subtype relation is, by definition, reflexive and transitive, it is immediate that $\sim$ is an equivalence on types.

The rules (REFL), (TRANS), (PAIR), (FUN) and (ATOM) are standard. The rule (TOP) introduces a universal type, and when combined with (COMPL), provides the dual notation of the bottom type, Bot, which is a subtype of any other type.

The rule (OK) allows us, together with (UNIONR) and (TRANS), to derive that each of Int, PairVal, Fun and Atom is a subtype of Ok i.e any normal form which is of these types is a value. The rule (DISJ) expresses the disjointness of certain types using the complement operator, for example Int, Pair, Fun, Atom, as well as distinct atoms 'a', 'b' and so on, are provably disjoint from one another. This is extended in (COMPR) - if A is disjoint from B , then their complements must be contained in the other. These two rules ensure that our complement operator behaves like an order-reversing involution, up to equivalence.

In the rule (PAIRC), we have Ok instead of Top in the second pair to mirror the asymmetric definition of $\llbracket (A, B) \rrbracket$. It is not necessarily true that $(\text{Top}, B^c) \sqsubseteq (A, B)^c$ since, if it was, we would be able to derive for example $(\exists (\lambda x. x), \text{div}) \in \llbracket (\text{Top}, \text{Int}^c) \rrbracket \sqsubseteq \llbracket (\text{Ok}^c, \text{Int}^c) \rrbracket$, but we know that $(\exists (\lambda x. x), \text{div}) \in \llbracket (\text{Ok}^c, \text{Int}) \rrbracket$.

The rules (UNIONL) and (UNIONR) follow naturally from a set theoretic perspective. They explain the union type as a least upper bound on types.

3.2 Typing Relation

We view the type system as a formal mechanism for establishing the truth of properties of the form “term M either diverges, or reduces to a normal form of the type A ”, written $M : A$.

Definition 3.4 (Typing Formula). A *typing formula* (or simply *typing*) is an expression of the form $M : A$, where M is a term, and A is a type. In particular, if M is a term variable, i.e. the typing formula has the form $x : A$, we call this a *variable typing*. The free variables of a typing formula $M : A$ are just $\text{FV}(M)$. We extend the notion to sets of formulas, writing $\text{FV}(\Delta)$ for $\bigcup_{M:A \in \Delta} \text{FV}(M)$.

The type system allows for proving sequents of the form $\Gamma \vdash \Delta$ with meaning, informally: if all the formulas in Γ are true then some formula in Δ is true³.

Definition 3.5 (Typing Judgement and Derivability). A *typing judgement* is an assertion of the form $\Gamma \vdash \Delta$, where Γ and Δ are possibly empty, finite sets of typing formulas. A type judgement is

³We will not formalise this interpretation directly, and instead soundness will be given via progress and preservation theorems.

VAR $\frac{}{\Gamma, x : A \vdash x : A, \Delta}$	TOP $\frac{}{\Gamma \vdash M : \text{Top}, \Delta}$	NUM $\frac{}{\Gamma \vdash n : \text{Int}, \Delta}$	ATOM $\frac{}{\Gamma \vdash 'a' : 'a', \Delta}$
SUBL $\frac{\Gamma, M : B \vdash \Delta}{\Gamma, M : A \vdash \Delta} A \sqsubseteq B$	SUBR $\frac{\Gamma \vdash M : B, \Delta}{\Gamma \vdash M : A, \Delta} B \sqsubseteq A$	COMPL $\frac{\Gamma \vdash M : A, \Delta}{\Gamma, M : A^c \vdash \Delta}$	COMPR $\frac{\Gamma, M : A \vdash \Delta}{\Gamma \vdash M : A^c, \Delta}$
BINOP $\frac{\Gamma \vdash M : \text{Int}, \Delta \quad \Gamma \vdash N : \text{Int}, \Delta}{\Gamma \vdash M \otimes N : \text{Int}, \Delta}$	RELOP $\frac{\Gamma \vdash M : \text{Int}, \Delta \quad \Gamma \vdash N : \text{Int}, \Delta}{\Gamma \vdash M \bowtie N : \text{Bool}, \Delta}$		
FIX $\frac{\Gamma, x : A \vdash M : A, \Delta}{\Gamma \vdash \text{fix } x. M : A, \Delta} x \notin \text{FV}(\Gamma, \Delta)$	PAIR $\frac{\Gamma \vdash M : A, \Delta \quad \Gamma \vdash N : B, \Delta}{\Gamma \vdash (M, N) : (A, B), \Delta}$		
ABS $\frac{\Gamma, x : A \vdash M : B, \Delta}{\Gamma \vdash \lambda x. M : A \rightarrow B, \Delta} x \notin \text{FV}(\Gamma, \Delta)$	APP $\frac{\Gamma \vdash M : B \rightarrow A, \Delta \quad \Gamma \vdash N : B, \Delta}{\Gamma \vdash M N : A, \Delta}$		
MATCH $\frac{\Gamma \vdash M : \bigvee_{i=1}^k p_i \theta_i, \Delta \quad \Gamma, \theta_i, M : p_i \theta_i \vdash N_i : A, \Delta \quad (\forall i \in [1, k])}{\Gamma \vdash \text{match } M \text{ with } \{\bigvee_{i=1}^k p_i \mapsto N_i\} : A, \Delta} \quad \forall i \in [1, k]. \text{PatSubst}(p_i, \theta_i) \wedge \text{FV}(p_i) \cap \text{FV}(\Gamma, \Delta) = \emptyset$			
OPe $\frac{\Gamma, M : \text{Int}, N : \text{Int} \vdash \Delta}{\Gamma, M \otimes N : \text{Ok} \vdash \Delta}$	RELOPe $\frac{\Gamma, M : \text{Int}, N : \text{Int} \vdash \Delta}{\Gamma, M \bowtie N : \text{Ok} \vdash \Delta}$	APPe $\frac{\Gamma, M : \text{Fun} \vdash \Delta}{\Gamma, M N : \text{Ok} \vdash \Delta}$	
MATCHe $\frac{\Gamma, M : \bigvee_{i=1}^k (p_i [\text{Ok}/x \mid x \in \text{FV}(p_i)]) \vdash \Delta}{\Gamma, \text{match } M \text{ with } \{\bigvee_{i=1}^k p_i \mapsto N_i\} : \text{Ok} \vdash \Delta}$	PAIRe $\frac{\Gamma, M : \text{Ok}, N : \text{Ok} \vdash \Delta}{\Gamma, (M, N) : \text{Ok} \vdash \Delta}$		

Fig. 2. Two-Sided Type System.

said to be *derivable* just if it is possible to construct a proof tree concluding the judgement using the rules of Figure 2. Excepting the (VAR) rule, the conclusion of each rule distinguishes a single typing, which is called the *principal formula* of the rule.

The side condition on the (MATCH) rule requires that $\text{PatSubst}(p, \theta)$, which holds just if θ is a pattern type substitution with $\text{dom}(\theta) = \text{FV}(p)$ and, for all types A in the range of θ , $A \sqsubseteq \text{Ok}$.

Two-sided type systems are construed as a kind of sequent calculus for typing formulas. That is, the rules of the proof system can be divided into those that explain how to *verify* a formula of a certain shape on the right-hand side of the sequent, and those that explain how to *refute* a formula of a certain shape on the left.

Since the right rules explain how to verify a typing, they are just the familiar rules of a traditional type system, but generalised slightly to allow for an arbitrary context Δ on the right (though one could instead consider an intuitionistic sequent calculus and drop this generalisation). Therefore

$$\begin{array}{c}
\text{WEAKL} \quad \frac{\Gamma \vdash \Delta}{\Gamma, M : A \vdash \Delta} \quad \text{WEAKR} \quad \frac{\Gamma \vdash \Delta}{\Gamma \vdash M : A, \Delta} \quad \text{PRJi} \quad \frac{\Gamma \vdash M : (A_1, A_2), \Delta}{\Gamma \vdash \pi_i(M) : A_i, \Delta} \quad A_1, A_2 \sqsubseteq \text{Ok} \quad \text{PRJEi} \quad \frac{\Gamma, M : \text{PairVal} \vdash \Delta}{\Gamma, \pi_i(M) : \text{Ok} \vdash \Delta} \\
\\
\text{If} \quad \frac{\Gamma \vdash M : \text{Bool}, \Delta \quad \Gamma \vdash N : A, \Delta \quad \Gamma \vdash P : A, \Delta}{\Gamma \vdash \text{if } M \text{ then } N \text{ else } P : A, \Delta} \quad \text{IfE} \quad \frac{\Gamma, M : \text{Bool} \vdash \Delta}{\Gamma, \text{if } M \text{ then } N \text{ else } P : \text{Ok} \vdash \Delta}
\end{array}$$

Fig. 3. Admissible rules for weakening, projections and conditionals.

we will not explain the standard rules (TOP), (NUM), (ATOM), (SUBR), (BINOP), (RELOP), (FIX), (PAIR), (ABS) and (APP).

The Match Rule. In the (MATCH) rule, we abuse notation by considering a pattern-type substitution θ , which is formally a finite map from term variables to types, also as a set of typings $\{x : A \mid \theta(x) = A\}$. The rule allows free choice of a pattern substitution θ , so long as it satisfies the side condition $\text{PatSubst}(p, \theta)$. This requires that any type $\theta(x)$ chosen as the type of a free variable x of pattern p provably does not contain any stuck term. This is essential for soundness, because otherwise it could be that the scrutinised term M could go wrong even when $\vdash M : \bigvee_{i=1}^k p_i \theta_i$, for example the judgement $\vdash \text{match } (1, 2) \text{ } 3$ with $\{x \mapsto 1\} : \text{Int}$ would be provable.

To understand the mechanics of the (MATCH) rule, let us first observe that all the rules of Figure 3 are admissible. Combining (MATCH) and (WEAKL), it follows that the less expressive, but perhaps more familiar, one-sided rule for matching is also admissible.

$$\begin{array}{c}
\text{MATCH}_0 \\
\frac{\Gamma \vdash M : \bigvee_{i=1}^k \theta_i p_i \quad \Gamma, \theta_i \vdash N_i : A \ (\forall i \in [1, k]) \quad \forall i \in [1, k]. \text{PatSubst}(p_i, \theta_i) \quad \wedge \text{FV}(p_i) \cap \text{FV}(\Gamma, \Delta) = \emptyset}{\Gamma \vdash \text{match } M \text{ with } \{\bigvee_{i=1}^k p_i \mapsto N_i\} : A}
\end{array}$$

This simpler match rule is sufficient when every case of the match produces a conclusion of the desired type A , such as the following instance with $\theta_1 = \emptyset$, $\theta_2 = [\text{Int}/y]$ and $\Gamma = \{x : ('data, \text{Int})\}$:

$$\begin{array}{c}
\text{VAR} \quad \frac{}{\Gamma \vdash x : ('data, \text{Int})} \\
\text{SUBR} \quad \frac{}{\Gamma \vdash x : 'err \vee ('data, \text{Int})} \quad \text{NUM} \quad \frac{}{\Gamma \vdash \underline{0} : \text{Int}} \quad \text{VAR} \quad \frac{}{\Gamma, y : \text{Int} \vdash y : \text{Int}} \\
\text{MATCH}_0 \quad \frac{}{\Gamma \vdash \text{match } x \text{ with } \{'err \mapsto \underline{0} \mid ('data, y) \mapsto y\} : \text{Int}}
\end{array}$$

However, this simpler form of rule is insufficient when there is some match case that produces a result of an incompatible type, yet the case does not apply. For example, if we were to modify the above program so that errors are propagated:

$$\text{match } x \text{ with } \{'err \mapsto 'err \mid ('data, y) \mapsto y\}$$

Here, we can use the extra left assumption of the second premise in the (MATCH) rule – specifically, $x : 'err$ – to refute that the case ever occurred. Writing A for the type $('data, \text{Int})$, we have:

$$\begin{array}{c}
\text{VAR} \quad \frac{}{\Gamma \vdash x : A, 'err : \text{Int}} \\
\text{SUBR} \quad \frac{\text{VAR} \quad \frac{}{\Gamma \vdash x : A}}{\Gamma \vdash x : 'err \vee A} \quad \text{COMPL} \quad \frac{\text{SUBL} \quad \frac{}{\Gamma, x : A^c \vdash 'err : \text{Int}}}{\Gamma, x : 'err \vdash 'err : \text{Int}} \quad \text{VAR} \quad \frac{}{\Gamma, y : \text{Int}, x : A \vdash y : \text{Int}} \\
\frac{}{\Gamma \vdash \text{match } x \text{ with } \{'err \mapsto 'err \mid ('data, y) \mapsto y\} : \text{Int}}
\end{array}$$

In the previous example, the guard expression x did not occur in the branches of the match expression. When it does, the (MATCH) rule provides the assumptions $M : p_i\theta_i$, describing the branch condition, to refine the type of the guard inside the branches. This is essential for typing the following innocent-looking variation, in which we assume only that the guard matches one of the patterns (and that the data is an integer in the second case), i.e. $\Gamma = \{x : \text{'err} \vee (\text{'data}, \text{Int})\}$, and again writing A to abbreviate $(\text{'data}, \text{Int})$:

$$\frac{\text{VAR} \frac{}{\Gamma \vdash x : \text{'err} \vee A} \quad \text{NUM} \frac{}{\Gamma, x : \text{'err} \vdash \underline{0} : \text{Int}} \quad \text{PRJ2} \frac{\text{VAR} \frac{}{\Gamma, y : \text{Int}, x : A \vdash x : A}}{\Gamma, y : \text{Int}, x : A \vdash \pi_2(x) : \text{Int}}}{\Gamma \vdash \text{match } x \text{ with } \{\text{'err} \mapsto \underline{0} \mid (\text{'data}, y) \mapsto \pi_2(x)\} : \text{Int}}$$

This way we can capture properties of the behaviour of matching expressions. For example, the following are all derivable types for $M = \lambda x. \text{match } x \text{ with } \{\text{'err} \mapsto \text{'err} \mid (\text{'data}, y) \mapsto y\}$:

$$\begin{array}{ll} \vdash M : \text{'err} \rightarrow \text{'err} & \vdash M : \text{'err} \vee (\text{'data}, \text{Int}) \rightarrow \text{'err} \vee \text{Int} \\ \vdash M : (\text{'data}, \text{Int}) \rightarrow \text{Int} & \vdash M : (\text{'err} \vee (\text{'data}, \text{Ok}))^c \rightarrow \text{Ok}^c \end{array}$$

The bottom-right typing allows us to certainly conclude that the application of this function to a term of type $(\text{'dat}, \text{Int})$, that is, where we have misspelled the atom, will not reduce to a value; e.g.

$$\vdash (\lambda x. \text{match } x \text{ with } \{\text{'err} \mapsto \text{'err} \mid (\text{'data}, y) \mapsto y\})(\text{'dat}, 3) : \text{Ok}^c$$

However, we will look in detail at how the system can be used for refuting that a term reduces to a value in the sequel.

The CompR Rule. The (COMPR) explains that to verify $M : A^c$, it suffices to refute $M : A$. This is because, verifying $M \in \llbracket A^c \rrbracket_*$ amounts to showing that M either diverges or reduces to a normal form of type A^c , and refuting $M : A$ amounts to showing that M does not reduce to a normal form of the type A . Since every term either diverges or normalises, it is straightforward to see that these are the exact same property.

Remark 3.6. A weakness of Ramsay and Walpole's work is that a complement operator was not supported by the type system. In particular, as discussed in [28], the rule (COMPR) was not sound, that is, its conclusion does not necessarily follow from its premise. This is because in that work, types are sets of *values*, and therefore the complement of a type A are those *values* that are not in the type A . Thus it does *not* suffice to refute $M : A$ in order to verify $M : A^c$, e.g. when M goes wrong. Ramsay and Walpole remedied this unsoundness by considering a system with an alternative semantics, inspired by success types [17]. However, this semantics has a fundamental shortcoming – verifying a typing no longer excludes the possibility that it may go wrong, so it is impossible to use this system to show that a given program cannot go wrong. Like success types, one can *only* certify that a program does *not* reduce to a value.

4 Refutation

The most interesting feature of the type system is the ability to deduce consequences of terms reducing to a normal form of a certain type and, ultimately, to refute typing formulas (by deducing absurdity as a consequence). As expressed by the soundness of the type system (Theorem 7.4), refuting a typing $M : \text{Ok}$, that is, proving a judgement $M : \text{Ok} \vdash$ with $M : \text{Ok}$ on the left-hand side and nothing on the right, gives a guarantee that M will *not* reduce to a value – in other words, M either diverges or it goes wrong. Thus, refutation of type assignments provides a mechanism for finding genuine bugs – with no false positives (assuming one counts divergence as a kind of bug).

BOTL $\frac{}{\Gamma, M : \text{Bot} \vdash \Delta}$	NUML $\frac{}{\Gamma, \underline{n} : \text{Int}^c \vdash \Delta}$	FUNL $\frac{}{\Gamma, \lambda x. M : \text{Fun}^c \vdash \Delta}$	ATOML $\frac{}{\Gamma, 'a : 'a^c \vdash \Delta}$
APPL $\frac{\Gamma \vdash M : B \multimap A, \Delta \quad \Gamma, N : B \vdash \Delta}{\Gamma, MN : A \vdash \Delta}$		FIXL $\frac{\Gamma, M : A \vdash x : A, \Delta}{\Gamma, \text{fix } x. M : A \vdash \Delta} \quad x \notin \text{FV}(\Gamma, \Delta)$	
SWAPL $\frac{\Gamma \vdash M : A^c, \Delta}{\Gamma, M : A \vdash \Delta}$	SWAPR $\frac{\Gamma, M : A^c \vdash \Delta}{\Gamma \vdash M : A, \Delta}$	IFL $\frac{\Gamma \vdash M : \text{Bool}, \Delta \quad \Gamma, N_i : A \vdash \Delta (\forall i)}{\Gamma, \text{if } M \text{ then } N_1 \text{ else } N_2 : A \vdash \Delta}$	
	PRJL1 $\frac{\Gamma \vdash M : (A^c, B), \Delta}{\Gamma, \pi_1(M) : A \vdash \Delta}$	PRJL2 $\frac{\Gamma \vdash M : (\text{Ok}, B^c), \Delta}{\Gamma, \pi_2(M) : B \vdash \Delta}$	
MATCHL $\frac{\Gamma \vdash M : \bigvee_{i=1}^k p_i \theta_i \quad \Gamma, M : p_i \theta_i, N_i : A \vdash \theta_i^c, \Delta (\forall i \in [1, k])}{\Gamma, \text{match } M \text{ with } \{ \lfloor_{i=1}^k p_i \mapsto N_i \} : A \vdash \Delta} \quad \text{FV}(p_1, \dots, p_k) \cap \text{FV}(\Gamma, \Delta) = \emptyset$			
PAIRL1 $\frac{}{\Gamma, (M, N) : \text{Pair}^c \vdash \Delta}$	PAIRLL $\frac{}{\Gamma, M : A \vdash \Delta}$	PAIRRL $\frac{\Gamma \vdash M : \text{Ok}, \Delta \quad \Gamma, N : B \vdash \Delta}{\Gamma, (M, N) : (A, B) \vdash \Delta}$	

Fig. 4. Derivable rules related to refutation.

The *left* rules of the system (those for which the principal formula in the conclusion of the rule is on the left of the turnstile) explain how to refute a typing formula. Using (OPE), one can directly refute that applying an arithmetic operation to an argument that is a function evaluates to a value, or, using (APPE), one can refute that applying a integer as if it were a function would evaluate.

TOP $\frac{}{\underline{3} : \text{Int}, x : \text{Bot} \vdash x : \text{Top}}$	
ABS $\frac{}{\underline{3} : \text{Int} \vdash \lambda x. x : \text{Fun}}$	NUM $\frac{}{\vdash \underline{3} : \text{Int}}$
COMPL $\frac{}{\underline{3} : \text{Int}, (\lambda x. x) : \text{Fun}^c \vdash}$	COMPL $\frac{}{\underline{3} : \text{Int}^c \vdash}$
SUBL $\frac{}{\underline{3} : \text{Int}, (\lambda x. x) : \text{Int} \vdash}$	SUBL $\frac{}{\underline{3} : \text{Fun} \vdash}$
OPE $\frac{}{\underline{3} \otimes (\lambda x. x) : \text{Ok} \vdash}$	APPE $\frac{}{\underline{3}(\underline{1}, \underline{2}) : \text{Ok} \vdash}$

Similarly, by (MATCHE) and (PAIRE), we can refute evaluation for e.g. scrutinizing a term that does not match any pattern, or trying to form a pair where one of the components does not evaluate.

However, in each case, it is the term itself that directly goes wrong. What about if we want to show that, for example, a redex $(\lambda x. M)N$ is contracted successfully, but that something in the body $M[N/x]$ will later get stuck; or that a match expression $\text{match 'a with } \{ 'a \mapsto N \mid 'b \mapsto P \}$ successfully evaluates one of the branches, but that evaluation of the branch gets stuck? For this purpose we introduce the left rules of Figure 4. However, we will later show that these rules are inherent to the system: each is already derivable from the rules of Figure 2.

The rule (BOTL) is dual to (TOP), and expresses that one may always refute that a term evaluates to a normal form inside the empty type. The rules (NUML), (FUNL), (ATOML) and (PAIRL1) express the self-evident facts that one can refute that a normal form lives inside the complement of its natural type.

The (APPL) rule gives a principle for refuting an application when the operator is, in fact, known to evaluate to a function. It explains that, to refute that application $M N$ evaluates to an A , we can show that M is a function which, to deliver an A , necessarily requires a B as input, i.e. $M : B \multimap A$, and then we only need to also show that the actual parameter N does not evaluate to a B . Note that, by the abbreviation, $B \multimap A$ is $B^c \rightarrow A^c$ so it literally states that every input outside of B will result in an output outside of A , which is equivalent to the contrapositive (see also Remark 3.2).

The rule (FIXL) gives a principle for refuting that a fixpoint $\text{fix } x. M$ evaluates to an A . To do so, we need only show that the argument being an A is a consequence of the body evaluating to an A . If the fixpoint is thought of as the limit of some chain of approximations, then the rule allows for working backwards along the chain, showing that when some better approximation is known to be outside of A , then the approximation preceding it was necessarily outside of A too.

The rule (MATCHL) gives a principle for refuting that a match expression $\text{match } M \text{ with } \{p_i \mapsto N_i\}_{i=1}^k$ evaluates to an A when control does get passed to the branches because the scrutinee matches some pattern. Note that, for convenience, when θ is used as a set of variable typings, we use the abbreviation θ^c for the set $\{x : A^c \mid x : A \in \theta\}$. To justify the refutation, we must show that M will evaluate to some value matching a pattern and then, for each branch $i \in [1, k]$, we refute that the branch returned an A by either:

- refuting $N_i : A$, thus proving directly that this branch did not return an A
- or, refuting $M : p_i \theta$, thus proving that this branch could not have been taken
- or, proving θ_i^c , thus showing that (although the branch may return an A) a consequence of the branch returning an A is that the assignment of types to the pattern variables is inconsistent with the actual matching θ .

Note that, since θ^c appears on the right of the judgement, it suffices only to show that *some* pattern variable is necessarily inconsistent with the actual matching.

The rules (PAIRLL) and (PAIRRL) gives the familiar principle that, to refute a pair (M_1, M_2) evaluates to a pair of an A_1 and an A_2 specifically, it suffices to refute that one component M_i evaluates to an A_i . However, to reflect the asymmetric definition of $\llbracket (A, B) \rrbracket$ (as discussed for rule (PAIRC) in Section 3.1), the (PAIRRL) rule imposes an additional constraint: if the refutation is based on the second component, the first must also reduce to a value, provided it does not diverge. This asymmetry also affects the rules (PRJLi) for refuting projections; to refute that a projection $\pi_i(M)$ yields a normal form of some type A , we show an incompatible component at position i , *and*, for second projection ($i = 2$) specifically, we must also ensure the first component would not go wrong.

Finally, the rule (IFL) allows us to refute that a conditional expression reduces to a value of type A by refuting that both of its branches reduce to a value of type A .

THEOREM 4.1. *All the rules of Figure 4 are derivable.*

PROOF. First observe that the swapping rules are derivable:

$$\begin{array}{c} \text{COMPL} \frac{\Gamma \vdash M : A^c \Delta}{\Gamma, M : (A^c)^c \vdash \Delta} \\ \text{SUBL} \frac{}{\Gamma, M : A \vdash \Delta} \end{array} \qquad \begin{array}{c} \text{COMPR} \frac{\Gamma, M : A^c \vdash \Delta}{\Gamma \vdash M : (A^c)^c, \Delta} \\ \text{SUBR} \frac{}{\Gamma \vdash M : A, \Delta} \end{array}$$

All remaining rules are obtained by reformulating a rule of Figure 2 using (COMPL), (COMPR), (SWAPL) and (SWAPR), e.g.:

(FIXL) Suppose $\Gamma, M : A \vdash x : A, \Delta$. Then by (COMPL) and (COMPR), $\Gamma, x : A^c \vdash M : A^c, \Delta$. Then $\Gamma \vdash \text{fix } x. M : A^c, \Delta$ by (FIX), and the result follows from (SWAPL).

□

Remark 4.2. Using the method from the proof of Theorem 4.1, we can derive from (ABS) a rule for refuting the typing of an abstraction:

$$\text{ABSL} \frac{\Gamma, M : B^c \vdash x : A^c, \Delta}{\Gamma, \lambda x. M : (A \rightarrow B)^c \vdash \Delta}$$

This rule is equivalent to the (CoABS) rule from Figure 5, which we will discuss further in Section 5.

One might also expect a left rule for refuting formulas of shape $M : A \rightarrow B$. Semantically, to refute $M : A \rightarrow B$ requires showing there exists some $N \in \llbracket A \rrbracket_*$ such that $MN \notin \llbracket B \rrbracket_*$. This suggests a rule like:

$$\text{ARR-LEFT} \frac{\Gamma \vdash N : A, \Delta \quad \Gamma, MN : B \vdash \Delta}{\Gamma, M : A \rightarrow B \vdash \Delta}$$

However, this rule would not be sound, since refuting $MN : B$ does not establish $MN \notin \llbracket B \rrbracket_*$ – it can be that MN diverges. Such a rule would also violate a nice property of our rules, namely that every term occurring in a premise of the rule is a subterm of a term occurring in the conclusion.

Remark 4.3. Our type system in Figure 2 contains just one left rule for each shape of stuck term, explaining the conditions under which such a term would get stuck. In contrast, in the two-sided systems of [28], there are a large number of primitive left rules, whose choice appears quite arbitrary and lack a convincing justification for their inclusion. For example, the two-sided system for PCF includes a rule (Dis) for refuting that a term M evaluates to a value of some type by means of verifying that it either diverges or evaluates to a value of some disjoint type, but they choose *not* to include a rule like our derived rule (PAIRL1). The reason given is that there is much overlap between these rules, for example they can both be used to show $(\underline{1}, \underline{2}) : \text{Nat} \vdash$. However, they are ultimately incomparable in expressive power, (Dis) is applicable even when the term subject M is not literally a pair, whereas (PAIRL1) applies even when the subject (P, Q) may not evaluate to a pair value, for example when P or Q goes wrong. In our system, however, both rules are derivable so there is no choice to make – the system of Figure 2 inherently contains both reasoning principles already.

Note, there are other differences between the rules of our system and analogous rules in the system of [28], which stem from the change to allow the typing of normal forms. For example, the rule (IfzL2) of [28] has an analogue in our (IfL). However, the latter has an additional premise $\Gamma \vdash M : \text{Bool}, \Delta$. Otherwise judgements such as if $\underline{2}$ then $\underline{3}$ else $\underline{4} : \text{Ok}^c \vdash$ would be derivable, and thus the system would be unsound.

4.1 Hébert's Examples

We will demonstrate some of these rules by examining the following sequence of three programs taken from Fred Hébert's introductory text on Erlang [13], where they are used to illustrate the power, and limitations, of success types and the Dialyzer for finding bugs. Although Erlang is a call-by-value language and ours is not, it makes no essential difference to these examples since our pattern matching is strict.

For the purpose of presenting the examples, a let construct is useful. As is well known, this can be defined as an abbreviation $\text{let } x = M \text{ in } N$ for the redex $(\lambda x. N) M$. Thus, it is easy to see that the following typing rules are derivable:

$$\begin{array}{c} \text{LET} \\ \frac{\Gamma \vdash M : B, \Delta \quad \Gamma, x : B \vdash N : A, \Delta}{\Gamma \vdash \text{let } x = M \text{ in } N : A, \Delta} \end{array} \quad \begin{array}{c} \text{LETL} \\ \frac{\Gamma, N : A \vdash x : B, \Delta \quad \Gamma, M : B \vdash \Delta}{\Gamma, \text{let } x = M \text{ in } N : A \vdash \Delta} \end{array}$$

The first of these rules should be familiar. The second explains how to refute that a let reduces to a normal form of type A . To do so, we can first derive necessary conditions on x , namely that it has some type B , in order for the term N to reduce to an A . Then we need only show that the term M , whose result is bound by x , *doesn't* satisfy this necessary type B .

The first program simply defines a wrapper around addition and misapplies it to an atom.

$$\text{let } op = \lambda xy. x + y \text{ in } op \underline{5} 'you$$

First observe that, if applying op to $\underline{5}$ and $'you$ were to yield a value, then it must be that op is *not* a function that *necessarily* needs its second argument to be an integer. We will express this property, that a curried function requires its second argument to be an integer, by the type $\text{Top} \rightarrow \text{Int} \multimap \text{Ok}$. We will let the reasoning by which we chose this type remain a mystery for the moment, except to say that this type represents those functions which, if they are given any first argument, either diverge or return a function that necessarily requires its argument to be an integer (to return a value). We will return to this topic in Section 5. For brevity, let $\Gamma := \{op : \text{Top} \rightarrow \text{Int} \multimap \text{Ok}\}$ and $\Delta := \{op : (\text{Top} \rightarrow \text{Int} \multimap \text{Ok})^c\}$.

$$\begin{array}{c} \text{VAR} \quad \frac{}{\Gamma \vdash op : \text{Top} \rightarrow \text{Int} \multimap \text{Ok}} \quad \text{TOP} \quad \frac{}{\Gamma \vdash \underline{5} : \text{Top}} \\ \text{APP} \quad \frac{}{\Gamma \vdash op \underline{5} : \text{Int} \multimap \text{Ok}} \quad \text{ATOML} \quad \frac{}{'you : ('you)^c \vdash \Delta} \\ \text{COMPR} \quad \frac{}{\vdash op \underline{5} : \text{Int} \multimap \text{Ok}, \Delta} \quad \text{SUBL} \quad \frac{}{'you : \text{Int} \vdash \Delta} \\ \text{APPL} \quad \frac{}{op \underline{5} 'you : \text{Ok} \vdash \Delta} \end{array}$$

Next, observe that, by definition, op *does* necessarily require its second argument to be an integer (in order to return a value), i.e. it is straightforward to derive: $\vdash \lambda xy. x + y : \text{Top} \rightarrow \text{Int} \multimap \text{Ok}$.

Then we can put these two derivations together to refute that this program reduces to a value:

$$\text{LETL} \quad \frac{op \underline{5} 'you : \text{Ok} \vdash op : (\text{Top} \rightarrow \text{Int} \multimap \text{Ok})^c \quad \text{COMPL} \quad \frac{\vdash \lambda xy. x + y : \text{Top} \rightarrow \text{Int} \multimap \text{Ok}}{\lambda xy. x + y : (\text{Top} \rightarrow \text{Int} \multimap \text{Ok})^c \vdash}}{\text{let } op = \lambda xy. x + y \text{ in } op \underline{5} 'you : \text{Ok} \vdash}$$

The second program in Hébert's sequence expresses the same problem more indirectly. There are defined functions *count* and *account* used for projecting out the numerical value of the transaction and the account identity respectively, and the transaction record is constructed using *money*. Then, instead of calling op directly, the second argument to op is itself a call to *account* which, when passed the result of *money*, returns an atom.

```

let money =  $\lambda xy. ('give, (x, y))$  in
let count =  $\lambda x. \text{match } x \text{ with } \{('give, (y, z)) \mapsto y\}$  in
let account =  $\lambda x. \text{match } x \text{ with } \{('give, (y, z)) \mapsto z\}$  in
let op =  $\lambda xy. x + y$  in
let tup = money  $\underline{5}$  'you in
  op (count tup) (account tup)

```

We observe that, for the body of the innermost let to reduce to a value, it *cannot* be that all of the following are true:

- (i) *op* requires its second argument to evaluate to an integer
- (ii) *and*, for *account* to return an integer, its argument must have evaluated to a value of shape $('give, (M, N))$, with *N* an integer
- (iii) *and*, *tup* is not of this shape

In other words, if this term evaluates, then one of (i)–(iii) is false. So let Δ_0 be the following encoding of this situation:

$$op : (\text{Top} \rightarrow \text{Int} \multimap \text{Ok})^c, \text{account} : (('give, (\text{Top}, \text{Int})) \multimap \text{Int})^c, \text{tup} : ('give, (\text{Top}, \text{Int}))$$

Then, it is straightforward to prove $op(\text{count } tup)(\text{account } tup) : \text{Ok} \vdash \Delta_0$ using (APPL), (APP), (COMPR) and (VAR). Moving outwards by one let, we find the definition of *tup* and we observe that for (the definition of) *tup* to evaluate to a tuple of shape $('give, (M, N))$, with *N* an integer, it *cannot* be that:

- (iii)' For *money* to return a tuple of shape $('give, (M, N))$, with *N* evaluating to an integer, requires its second argument to be an integer.

This is because *money* is not given an integer as second argument in the call in the definition of *tup*. We can prove this either by working backwards using (APPL) or we can exchange the two sides of the judgement using (SWAPL) and (COMPR), and prove the resulting judgement as we would in a traditional type system:

$$\frac{\text{COMPR} \frac{\text{money} : \text{Top} \rightarrow \text{Int}^c \rightarrow ('give, (\text{Top}, \text{Int}))^c \vdash \text{money} \underline{5} 'you : ('give, (\text{Top}, \text{Int}))^c}{\vdash \text{money} : (\text{Top} \rightarrow \text{Int} \multimap ('give, (\text{Top}, \text{Int})))^c, \text{money} \underline{5} 'you : ('give, (\text{Top}, \text{Int}))^c}}{\text{SWAPL} \frac{}{\text{money} \underline{5} 'you : ('give, (\text{Top}, \text{Int})) \vdash \text{money} : (\text{Top} \rightarrow \text{Int} \multimap ('give, (\text{Top}, \text{Int})))^c}}$$

In either case, we have that, for the innermost let to evaluate, we must have that at least one of (i), (ii) and (iii)' is false, so let us abbreviate this conclusion by Δ_1 :

$$\begin{aligned} &op : (\text{Top} \rightarrow \text{Int} \multimap \text{Ok})^c \\ &\text{account} : (('give, (\text{Top}, \text{Int})) \multimap \text{Int})^c \\ &\text{money} : (\text{Top} \rightarrow \text{Int} \multimap ('give, (\text{Top}, \text{Int})))^c \end{aligned}$$

Then we can combine the above derivations, using (LETL) and the weakening rule, to prove:

$$\frac{op(\text{count } tup)(\text{account } tup) : \text{Ok} \vdash \text{tup} : ('give, (\text{Top}, \text{Int})), \Delta_1 \quad \text{money} \underline{5} 'you : ('give, (\text{Top}, \text{Int})) \vdash \Delta_1}{\text{let } \text{tup} = \text{money} \underline{5} 'you \text{ in } op(\text{count } tup)(\text{account } tup) : \text{Ok} \vdash \Delta_1}$$

This, in turn, becomes the left premise of (LETL) for the let expression $\text{let } op = \lambda xy. x + y \text{ in } \dots$, and so on, and it remains only to show that the various local identifiers *do* satisfy properties (i)–(iii)', thus contradicting that the program as a whole reaches a value. Let us write Δ_2 for the environment

$\Delta_1 \setminus \{op : (\text{Top} \rightarrow \text{Int} \multimap \text{Ok})^c\}$, and P for $\text{let } tup = \text{money} _5 \text{'you in } op(\text{count } tup)(\text{account } tup)$, and noting that the judgement $\vdash \lambda xy. x + y : \text{Top} \rightarrow \text{Int} \multimap \text{Ok}$ can be proven as above:

$$\text{LETL} \frac{P : \text{Ok} \vdash \Delta_1 \quad \text{COMPL} \frac{\vdash \lambda xy. x + y : \text{Top} \rightarrow \text{Int} \multimap \text{Ok}, \Delta_2}{\lambda xy. x + y : (\text{Top} \rightarrow \text{Int} \multimap \text{Ok})^c \vdash \Delta_2}}{\text{let } op = \lambda xy. x + y \text{ in } P : \text{Ok} \vdash \Delta_2}$$

Eventually, by this process, we can refute that the program as a whole evaluates.

The final program in Hébert's sequence is one for which Dialyzer's automatic type inference fails, but nevertheless will crash at runtime. It is a small modification of the former program where, instead of two distinct functions *count* and *account*, there is a single function *item* whose behaviour is either like *count* or like *account* depending on a switch.

$$\begin{aligned} \text{item} &= \lambda x. \text{match } x \text{ with} \\ &\quad \left| \begin{array}{ll} ('count, ('give, (y, z))) & \mapsto y; \\ ('account, ('give, (y, z))) & \mapsto z; \end{array} \right. \end{aligned}$$

Since the program has essentially the same structure as the previous, we may adopt the same strategy for refuting that it can reach a value. Here we can deduce that, for the body of the innermost let to have reduced to a value, it *must not* be that all of the following are true:

- (i) *op* is a function that requires that its second argument evaluates to an integer
- (ii) and, *item* is a function that, to return an integer, requires an input that does not evaluate to a value of shape $('account, ('give, (P, Q)))$, with Q not an integer
- (iii) and, *tup* does evaluate to a value of shape $('give, (P, Q))$ with Q not an integer.

As previously, our argument proceeds by showing that, in fact, *all* of (i)–(iii) do hold, contradicting the assumption that the program did evaluate. The approach is similar to the previous two examples, but of particular interest is the change from separate *count* and *account* to a single function *item*. However, using our sophisticated (MATCHL) rule we can prove (ii):

$$\vdash \lambda x. \text{match } x \text{ with } \{ \dots \} : ('account, ('give, (\text{Ok}, \text{Int}^c \wedge \text{Ok})))^c \multimap \text{Int}$$

Note: we have to use $\text{Int}^c \wedge \text{Ok}$ to ensure that the type $('account, ('give, (\text{Ok}, \text{Int}^c \wedge \text{Ok})))$ is a subtype of Ok , according to the side-condition on the rule. To do so, we assume that the body of the function evaluates to an integer and show that the argument x therefore did not match the 'account branch without z being an integer. Of course, we could also show that it follows that x did not match the 'count branch without y being an integer, but this is not relevant to the overall argument. By (COMPR), it suffices to assume that x *does* match the 'account branch without z being an integer, and then use (MATCHL) to refute that the match expression evaluated to an integer.

$$x : ('account, ('give, (\text{Ok}, \text{Int}^c \wedge \text{Ok}))), \text{match } x \text{ with } \{ \dots \} : \text{Int} \vdash$$

By taking $\theta_1 = [\text{Int}^c \wedge \text{Ok}/y, \text{Ok}/z]$ and $\theta_2 = [\text{Ok}/y, \text{Int}^c \wedge \text{Ok}/z]$, this assumption on x , let us write Δ_x for short, ensures that we can prove that the match expression did evaluate some branch:

$$\Delta_x \vdash x : ('count, ('give, (\text{Int}^c \wedge \text{Ok}, \text{Ok}))) \vee ('account, ('give, (\text{Ok}, \text{Int}^c \wedge \text{Ok})))$$

Then it is straightforward to refute that some branch returned an integer, by showing that in each of the two branches, returning an integer is inconsistent with this assignment to the free pattern variables y and z (i.e. that θ_i^c follows in each case). For example, in the second branch:

$$\text{SUB} \frac{\text{VAR} \frac{}{x : ('account, ('give, (\text{Ok}, \text{Int}^c \wedge \text{Ok}))), z : \text{Int} \vdash y : \text{Ok}^c, z : \text{Int}}{x : ('account, ('give, (\text{Ok}, \text{Int}^c \wedge \text{Ok}))), z : \text{Int} \vdash y : \text{Ok}^c, z : (\text{Int}^c \wedge \text{Ok})^c}}{x : ('account, ('give, (\text{Ok}, \text{Int}^c \wedge \text{Ok}))), z : \text{Int} \vdash y : \text{Ok}^c, z : (\text{Int}^c \wedge \text{Ok})^c}$$

Having derived this type for *item*, the rest of the argument follows in the same way as the two previous examples, so that overall, we can refute that the program reduces to a value.

Remark 4.4. Dialyzer fails to certify this example automatically because it can only infer the (Erlang) type $(\text{'account'} \mid \text{'count'}, \{\text{'give'}, 5, \text{'you'}\}) \rightarrow \text{'you'} \mid 5$, in which the dependence between the atom 'account' in the input and the atom 'you' in the output has been lost. However, its analysis can be guided by extending the example with the following “overloaded type specification”, which explicitly enumerates the dependencies between the two calls and their return values, and then it can certify the bug.

$$\begin{aligned} & \text{-spec item}(\text{'account'}, \{\text{'give'}, 5, \text{'you'}\}) \rightarrow \text{'you'}; \\ & (\text{'count'}, \{\text{'give'}, 5, \text{'you'}\}) \rightarrow 5. \end{aligned}$$

5 The CoArrow Type

In the discussion of Hébert’s first example, we used the type $\text{Top} \rightarrow \text{Int} \multimap \text{Ok}$ to assert the property that a curried function requires its second argument to be an integer. This works because the type mixes the two kinds of arrows in a specific way: it is the type of those functions that, if you give them any first argument will either diverge or return a function that requires an integer in order to return a value.

However, it is not clear there is a general pattern to forming assertions of the form “is a curried function which, to return an A , requires that its n^{th} argument is a B ” using the necessity arrow. For example, *neither* of the following two types express that, to return a value, the first argument of a two-argument curried function must be an integer:

$\text{Int} \multimap \text{Top} \multimap \text{Ok}$. This is the type of functions that, if they return a function of type $\text{Top} \multimap \text{Ok}$, their argument must have been an integer. Hence, this property holds of any function that does not return a function, such as $\lambda x. \perp$.

$\text{Int} \rightarrow \text{Top} \multimap \text{Ok}$. This is the type of functions that, if they are given an integer as the first argument, guarantee to return a function, about which we have no further guarantees (if the function returns a value, then it’s argument must have had type Top , which is no information at all). An example is $\lambda xy. \perp$.

By unfolding the abbreviations, one can see that such a property *could* be expressed by the type $\text{Int}^c \rightarrow \text{Top} \rightarrow \text{Ok}^c$. However, it is not at all clear how to use this type with the (APPL) rule.

The problem is that the necessity arrow does not curry well. Apart from the difficulties illustrated above, this can be seen clearly in the proof theory. When we type a curried abstraction using the traditional arrow, the two uses of the (ABS) rule follow directly on one another, shown below left.

$$\begin{array}{c} \text{ABS} \frac{x : A, y : B \vdash M : C}{x : A \vdash \lambda y. M : B \rightarrow C} \\ \text{ABS} \frac{x : A \vdash \lambda y. M : B \rightarrow C}{\vdash \lambda xy. M : A \rightarrow B \rightarrow C} \end{array} \qquad \text{ABN} \frac{\lambda y. M : B \multimap C \vdash x : A}{\vdash \lambda xy. M : A \multimap B \multimap C}$$

In the conclusion of the rule, the typing formula is of shape $\lambda z. P : A_1 \rightarrow A_2$ and occurs on the right of the judgement and in the premise we have a formula of exactly the same shape. However, this is not the case for the abstraction rule for the necessity function type (ABN) (which was included in [28] as a primitive rule, but is easy to derive in our system), shown above on the right. This rule doesn’t apply to the judgement $\lambda y. M : B \multimap C \vdash x : A$. Indeed, we can only proceed by subsumption on the left, which is not helpful.

Although the necessity arrow $A \multimap B$ appears natural, it is not, on its own, quite the right concept through which to properly understand the property that a function requires a certain property of its argument. In fact, a better choice is to use the *complement* of the necessity arrow.

$$\begin{array}{c}
\text{CoAbs} \\
\frac{\Gamma, M : A \vdash x : B, \Delta}{\Gamma, \lambda x. M : B \not\vdash A \vdash \Delta}
\end{array}
\qquad
\begin{array}{c}
\text{CoApp} \\
\frac{\Gamma, M : B \not\vdash A \vdash \Delta \quad \Gamma, N : B \vdash \Delta}{\Gamma, MN : A \vdash \Delta}
\end{array}$$

Fig. 5. Derivable abstraction and application rules *a la* bi-intuitionistic logic.

At first this may seem strange, because the complement of the necessity arrow, $(A \multimap B)^c$, is hardly a function type at all. This type, viewed as a set under Definition 3.1, contains integers and pairs and so on – even stuck terms belong to this type! However, *refuting* that a term has this type amounts to showing that the term is a function that requires an A as input if it is to return a B . Moreover, we can chain this type to describe the properties of curried functions in a meaningful way. Let us call this function type the *coarrow*, or *coto* for short. Let us also introduce the abbreviation: $A \not\vdash B := (A \multimap B)^c$. Then the typing rule (CoAbs) of Figure 5 is derivable. Using the coarrow, we can type curried abstractions as easily as we type them with the traditional arrow type, because of the implicit negation that surrounds the type of the returned function.

This derivation also helps to clarify the meaning of a type of shape $A \not\vdash B \not\vdash C$. We can *refute* that a curried function $\lambda xy. M$ satisfies this type whenever we can show that M evaluating to a C implies that *either* x must have been an A *or* y must have been a B . In particular, by using Bot, we can therefore state *the negation of* properties of the form desired above. For example:

Bot $\not\vdash$ Int $\not\vdash$ Ok. This is the type of normal forms that are *not* functions that, to return a value, require their second argument to be an integer. It is easy to see, by unfolding the abbreviations, that it is subtype equivalent to the complement of the type $\text{Top} \rightarrow \text{Int} \multimap \text{Ok}$, which we used in Hébert’s first example.

Int $\not\vdash$ Bot $\not\vdash$ Ok. This is the type of normal forms that are *not* functions that, to return a value, require their first argument to be an integer. Unfolding the abbreviations, this type is subtype equivalent to the complement of that we suggested above: $\text{Int}^c \rightarrow \text{Top} \rightarrow \text{Ok}^c$.

Refuting that a term has one of these types, therefore, verifies that it satisfies the corresponding property of functions. Moreover, the (CoApp) variant of (AppL), of Figure 5, is easily derivable.

Hopefully the reader can agree that the proof theory works out quite smoothly (see also the examples below), but the nature of this type is so unusual – the ‘function type’ $\text{Top} \not\vdash \text{Bot}$ doesn’t contain any functions – that one might want some independent confirmation that it is really a natural choice.

In fact, the logical counterpart to this type is the key feature in the extension of intuitionistic logic to bi-intuitionistic logic (also called Heyting-Brouwer [29], or subtractive logic [9]). As is well known, the usual function type $A \rightarrow B$ is related to the propositional implication, A implies B , by the Curry-Howard Correspondence. Propositional implication has a natural dual, which is variously called co-implication, subtraction or pseudo-difference. Where implication can be characterised by the adjunction below left (whose forward direction corresponds to the usual (Abs) rule), co-implication can be defined by the below right⁴, whose correspondence with our (CoAbs) rule is immediately clear.

$$A \wedge B \vdash C \quad \text{iff} \quad A \vdash B \rightarrow C \qquad B \not\vdash A \vdash C \quad \text{iff} \quad A \vdash B \vee C$$

⁴Some authors prefer to define coimplication rather as the converse to this relation, i.e. with the A and B exchanged, see the second footnote of [31].

Example 5.1. The following example is a good illustration of how to use the coarrow. Observe that we can refute that the *twice* function, $\lambda f x. f (f x)$, can be assigned the type $(\text{Int} \nrightarrow \text{Int}) \nrightarrow \text{Int} \nrightarrow \text{Int}$. Writing Δ for $\{x : \text{Int}, f : \text{Int} \nrightarrow \text{Int}\}$, we have the derivation:

$$\begin{array}{c}
 \text{VAR} \frac{}{f : \text{Int} \nrightarrow \text{Int} \vdash \Delta} \quad \text{VAR} \frac{}{x : \text{Int} \vdash \Delta} \\
 \text{CoAPP} \frac{\text{VAR} \frac{}{f : \text{Int} \nrightarrow \text{Int} \vdash \Delta} \quad \text{CoAPP} \frac{\text{VAR} \frac{}{x : \text{Int} \vdash \Delta}}{f x : \text{Int} \vdash \Delta}}{f (f x) : \text{Int} \vdash \Delta} \\
 \text{CoABS} \frac{}{\lambda x. f (f x) : \text{Int} \nrightarrow \text{Int} \vdash f : \text{Int} \nrightarrow \text{Int}} \\
 \text{CoABS} \frac{}{\lambda f x. f (f x) : (\text{Int} \nrightarrow \text{Int}) \nrightarrow \text{Int} \nrightarrow \text{Int} \vdash}
 \end{array}$$

From the conclusion of this refutation, we are assured that, if the *twice* function returns an integer, it must be that *either*:

- (i) its second argument was an integer
- (ii) *or*, its first argument was not a function that required an integer in order to return one

Another way to read this implication is that, if the *twice* function returns an integer and its second argument was *not* an integer, then it must be that its first argument didn't require an integer in order to return one.

Moreover, it is straightforward to show that $\lambda x. x + \underline{2}$ is a function that, to return an integer, requires an integer as input. That is, we can *refute* that it has type $\text{Int} \nrightarrow \text{Int}$. Therefore, we can prove that $(\lambda f x. f (f x)) (\lambda x. x + \underline{2})$ really requires an integer in order to evaluate to an integer. That is, we have the following refutation:

$$\text{CoAPP} \frac{\lambda f x. f (f x) : (\text{Int} \nrightarrow \text{Int}) \nrightarrow \text{Int} \nrightarrow \text{Int} \vdash \quad \lambda x. x + \underline{2} : \text{Int} \nrightarrow \text{Int} \vdash}{(\lambda f x. f (f x)) (\lambda x. x + \underline{2}) : \text{Int} \nrightarrow \text{Int} \vdash}$$

6 Metatheory of Subtyping

The rules of the subtype relation are natural from our understanding of types as certain sets of normal forms, so it is not surprising that these sets form a model of the theory, as revealed by Theorem 6.1. The contrapositive of the theorem allows us to use the set-theoretic model to exclude that certain subtype inequalities are derivable. For example, we cannot have $A \rightarrow B \sqsubseteq \text{Int}$ for any types A and B , nor can we derive $A \rightarrow B \sqsubseteq (B \rightarrow C)^c$ for any B and C , since the set on the left always contains $\lambda x. \underline{\text{div}}$ but the set on the right does not.

THEOREM 6.1 (SOUNDNESS OF SUBTYPING). *If $A \sqsubseteq B$ then $\llbracket A \rrbracket \subseteq \llbracket B \rrbracket$.*

However the axiomatisation itself is not straightforward to analyse because of the unrestricted transitivity rule and the general rules regarding complementation. Yet it is essential to understand certain invariants of this relation to properly argue the soundness of the type system. For example, for soundness, we require that whenever $A \rightarrow B \sqsubseteq C \rightarrow D$ is derivable, then it must be that already $C \sqsubseteq A$ and $B \sqsubseteq D$. If this inequality is derived using the (FUN) rule, then all is well, but as usual the unrestricted transitivity rule allows for other more convoluted proofs. For example, we can prove $\text{Int} \rightarrow \text{Int} \sqsubseteq \text{Int} \rightarrow \text{Ok}$ using transitivity with the mid-point $(\bigvee_{i=1}^2 (\text{Int}^{cc} \rightarrow (\text{Int} \vee 'a \vee 'b_i))^c)^c$. Clearly, there is no need to invoke transitivity at all in the proof of this inequality, but even supposing we decide to invoke transitivity, this chosen mid-point is excessively complicated – through a combination of the union and complementation rules it can be seen equivalent to the simpler $\text{Int} \rightarrow (\text{Int} \vee 'a)$.

Thus we are led to present an alternative characterisation of the subtyping relation in which the space of possible proofs is significantly more constrained, although, as we shall see, the provable

REFL $\frac{}{A \leq A}$	TOP $\frac{}{A \leq \text{Top}}$	OKR $\frac{A \leq B}{A \leq \text{Ok}} \quad B \in \mathcal{B}$	ATOM $\frac{}{a \leq \text{Atom}}$	COMPRATOM1 $\frac{}{b \leq a^c} \quad a \neq b$	COMPLATOMS $\frac{}{\text{Atom}^c \leq a^c}$
	COMPRATOM2 $\frac{A \leq B}{A \leq a^c} \quad B \in \{\text{Int, Pair, Fun}\}$		COMPRINT $\frac{A \leq B}{A \leq \text{Int}^c} \quad B \in \{\text{Pair, Fun, Atom}\}$		
	COMPRPAIR $\frac{A \leq B}{A \leq (B, C)^c} \quad B \in \{\text{Int, Fun, Atom}\}$		COMPRARR $\frac{A \leq B}{A \leq (D \rightarrow E)^c} \quad B \in \{\text{Int, Pair, Atom}\}$		
COMPRATOMS $\frac{A \leq B}{A \leq \text{Atom}^c} \quad B \in \{\text{Int, Pair, Fun}\}$		COMPRPAIRL $\frac{A \leq C^c}{(A, B) \leq (C, D)^c}$		COMPRPAIRR $\frac{A \leq \text{Ok} \quad B \leq D^c}{(A, B) \leq (C, D)^c}$	
COMPRUN $\frac{A \leq A_i^c \ (\forall i \in [1, k])}{A \leq (\bigvee_{i=1}^k A_i)^c}$	COMPRC $\frac{A \leq B}{A \leq (B^c)^c}$	PAIR $\frac{A \leq A' \quad B \leq B'}{(A, B) \leq (A', B')}$		FUN $\frac{A' \leq A \quad B \leq B'}{A \rightarrow B \leq A' \rightarrow B'}$	
UNIONR $\frac{A \leq B_j}{A \leq \bigvee_{i=1}^k B_i} \quad j \in [1, k]$	UNIONL $\frac{B_i \leq A \ (\forall i \in [1, k])}{\bigvee_{i=1}^k B_i \leq A}$	COMPLTOP $\frac{}{\text{Top}^c \leq B}$		COMPLPAIR $\frac{A^c \leq A'^c \quad B^c \leq B'^c}{(A, B)^c \leq (A', B')^c}$	
COMPLARR $\frac{A'^c \leq A^c \quad B^c \leq B'^c}{(A \rightarrow B)^c \leq (A' \rightarrow B')^c}$	COMPLOK $\frac{B^c \leq A}{\text{Ok}^c \leq A} \quad B \in \mathcal{B}$	COMPLUN $\frac{B_j^c \leq A}{(\bigvee_{i=1}^k B_i)^c \leq A} \quad j \in [1, k]$		COMPLC $\frac{A \leq B}{(A^c)^c \leq B}$	

Fig. 6. Alternative rules for subtyping.

inequalities are the same. The rules of Figure 6 are obtained from the the rules of the original subtyping relation by ‘baking in’ transitivity, and enforcing a more principled use of the (COMPL) and (COMPR) rules.

Definition 6.2 (Alternative Subtyping). The alternative subtyping relation $A \leq B$ is defined by the rules of Figure 6. Here $\mathcal{B}$ abbreviates the set $\{\text{Int, PairVal, Fun, Atom}\}$.

The motivation for restricting the complementation rules is as follows. When constructing a proof of $A \sqsubseteq B^c$, bottom-up, one may sometimes find a choice between using disjointness (DISJ), (PAIRC) or the (COMPR) rule. If the (COMPR) rule is chosen, then there are many choices with how to conclude the subproof of $B \sqsubseteq A^c$. For example, one may use disjointness with A , or some rule that exploits the structure of B , or even using (COMPR) again to uselessly return to the original conclusion. In our alternative system of Figure 6, we split (COMPR) into a large number of cases, and we restrict that proofs should follow a particular strategy based on the syntax of B . In particular, in most cases when the form of the judgement $A \sqsubseteq B^c$ would call for a use of (COMPR) in the original system, the alternative system will insist that this should be followed by some rule exploiting the structure of B , followed by a use of (COMPL) to ‘undo’ the rearrangement. For example, the rule (COMPRUN) of the alternative system corresponds to the following derivation in the original:

$$\begin{array}{c}
\text{COMPL} \frac{A \sqsubseteq B_1^c}{B_1 \sqsubseteq A^c} \quad \dots \quad \text{COMPL} \frac{A \sqsubseteq B_k^c}{B_k \sqsubseteq A^c} \\
\text{UNIONL} \frac{}{\bigvee_{i=1}^k B_i \sqsubseteq A^c} \\
\text{COMPR} \frac{}{A \sqsubseteq (\bigvee_{i=1}^k B_i)^c}
\end{array}$$

The alternative characterisation of subtyping makes it much easier to prove the invariants we need in order to establish the soundness of the type system. For example, a brief inspection of the rules reveals that (FUN) is now the *only* way to derive $A \rightarrow B \leq C \rightarrow D$. In particular, there is no transitivity rule and the complementation rules are restricted by the syntax of the complemented type. However, it is straightforward to show that the original rules are admissible, and then equivalence with the original system follows by induction.

THEOREM 6.3 (EQUIVALENCE OF SUBTYPING RELATIONS). $A \sqsubseteq B$ iff $A \leq B$

As well as making it easier to establish certain invariants, the alternative characterisation gives us a straightforward approach to argue its decidability.

THEOREM 6.4 (DECIDABILITY). *The problem, given A and B determine derivability of $A \sqsubseteq B$, is decidable.*

PROOF. Let A and B be types given as input. By the equivalence theorem, it suffices to decide $A \leq B$. Let $\text{SubExp}(A)$ be the set of all subexpressions⁵ of the type A and $\text{SubExp}(B)$ be the set of all subexpressions of the type B . Then consider the following set of types:

$$\mathcal{A}_{A,B} := \text{SubExp}(A) \cup \text{SubExp}(B) \cup \{\text{Top}, \text{Int}, \text{Pair}, \text{PairVal}, \text{Fun}, \text{Atom}, \text{Ok}\}$$

This set consists of all the finitely many types that occur as subexpressions of the input types A and B as well as certain fixed types such as Top , Int and so on. Then define the following finite set of types:

$$\mathcal{U}_{A,B} := \mathcal{A}_{A,B} \cup \{D^c \mid D \in \mathcal{A}_{A,B}\}$$

One can see, by inspection of the alternative subtyping rules in Figure 6, that whenever a rule instance is concluded by some inequality $C \leq D$, and C and D are in $\mathcal{U}_{A,B}$, then every type occurring in the premises of the rule instance are also contained in $\mathcal{U}_{A,B}$. Hence, in any proof of $A \leq B$ it must be that every subproof is concluded by an inequality of shape $C \leq D$, where both C and D are in $\mathcal{U}_{A,B}$, of which there are only finitely many. Consequently, a bottom-up proof search that avoids cycles will decide $A \leq B$. $\square$

7 Metatheory of Type System

For developing the metatheory of the type system, we first simplify the situation. Two-sided type systems have arbitrary typing formulas on the left- and right-hand sides of the judgement. However, we show that it is possible to reformulate the type system so that arbitrary typings are needed only on the right (though we may still need any number of them).

Definition 7.1 (One-Sided System). Judgements of the one-sided system have shape $\Gamma \vdash \Delta$ in which every typing in Γ is a variable typing, of shape $x : A$. The rules are those of the two-sided system but with (SUBL) removed, and (COMPL), (MATCH), (OPE), (RELOPE), (PAIRR), (APPE) and (MATCHE) replaced by the rules of Figure 7.

⁵As usual, we consider A to be a subexpression of itself.

$$\begin{array}{c}
\text{COMP} \\
\frac{\Gamma, x : A \vdash \Delta}{\Gamma \vdash x : A^c, \Delta} \\
\\
\text{MATCH} \\
\frac{\Gamma \vdash M : \bigvee_{i=1}^k p_i \theta_i, \Delta \quad \Gamma, \theta_i \vdash M : (p_i \theta_i)^c, N_i : A, \Delta \ (\forall i)}{\Gamma \vdash \text{match } M \text{ with } \{ \lfloor_{i=1}^k p_i \mapsto N_i \} : A, \Delta} \\
\\
\text{OPe} \quad \frac{\Gamma \vdash M : \text{Int}^c, N : \text{Int}^c, \Delta}{\Gamma \vdash M \bowtie N : \text{Ok}^c, \Delta} \quad
\text{RELOPe} \quad \frac{\Gamma \vdash M : \text{Int}^c, N : \text{Int}^c, \Delta}{\Gamma \vdash M \otimes N : \text{Ok}^c, \Delta} \quad
\text{PAIRE} \quad \frac{\Gamma \vdash M : \text{Ok}^c, N : \text{Ok}^c, \Delta}{\Gamma \vdash (M, N) : \text{Ok}^c, \Delta} \\
\\
\text{APPE} \quad \frac{\Gamma \vdash M : \text{Fun}^c, \Delta}{\Gamma \vdash MN : \text{Ok}^c, \Delta} \quad
\text{MATCHE} \quad \frac{\Gamma \vdash M : (\bigvee_{i=1}^k p_i [\text{Ok}/x | x \in \text{FV}(p_i)])^c, \Delta}{\Gamma \vdash \text{match } M \text{ with } \{ \lfloor_{i=1}^k p_i \mapsto N_i \} : \text{Ok}^c, \Delta}
\end{array}$$

Fig. 7. One-sided typing rules (differences).

The idea is as follows. It is easy to see that, the one-sided and two-sided versions of (MATCH), (OPe), (RELOPe), (PAIRE), (APPE) and (MATCHE) are inter-derivable in the two-sided system by bracketing rule uses with (COMPL) and (COMPR). Therefore, one can suppose that the original, two-sided system uses rules in Figure 7 instead. In that case, a short inspection of the remaining two-sided rules shows that the only way to conclude a typing on the left-hand side is by (SUBL) or (COMPL), and thus the latter must eventually be used. This implies that to refute $M : A$, we must have already verified $M : B$ on the right, for some B such that $A \sqsubseteq B^c$. Therefore, every refutation $M : A$ corresponds to a verification of some $M : B$, for which we did not need any left rules. Formally, the proof is by induction.

THEOREM 7.2 (EQUIVALENCE). *Let us write $\Gamma \vdash_2 \Delta$ for provability in the two-sided system and $\Gamma \vdash_1 \Delta$ for provability in the one-sided system. Then both of the following:*

- (i) *If $\Gamma \vdash_2 \Delta$ then $\vdash_1 \Gamma^c, \Delta$.*
- (ii) *If $\Gamma \vdash_1 \Delta$ then $\Gamma \vdash_2 \Delta$.*

Note: a judgement like $x : A, x : A^c \vdash$, although provable in the two-sided system is *not* provable in the one-sided system. On the other hand, the theorem shows that the “equivalent” judgement $\vdash x : A, x : A^c$ is provable in the one-sided system. The asymmetry stems from the fact that the one-sided rules have (COMP) that may faithfully simulate (COMPR) on variable typings, but no rule to faithfully simulate (COMPL) on variable typings.

This is a useful simplification, but there still remains the problem of multiple conclusions on the right. This is a significant barrier to a standard progress and preservation style argument for the soundness of the type system. Consider progress for a judgement of shape $\Gamma \vdash M : \text{Ok}, \Delta$. When Γ and Δ are arbitrary, and not only variable typings, it is not true that M in normal form will be a value. This is the case in, for example, the judgement:

$$\vdash \underline{z}(\lambda z. z) : \text{Ok}, \lambda x. x : \text{Int} \rightarrow \text{Int}$$

This judgement expresses the property: *either $\underline{z}(\lambda z. z)$ diverges or yields an integer value, or $\lambda x. x$ is an integer function.* Since the second conclusion is true, the truth of the judgement as a whole simply does not depend on the drivel in the first conclusion, and so we cannot expect to be able to rely on anything about it.

Remark 7.3. In fact, this highlights a deficiency in the original work. Due to this difficulty, progress and preservation in [28] was proven only for judgements satisfying the *single-subject constraint*, that is, in which at most one term in the judgement is not a variable. Thus, they establish soundness only for fragments of their two-sided type systems.

Here, rule (APP) is applied at the root of the derivation tree to obtain $\underline{2}(\lambda z. z)$. However, the validity of the judgement does not rely on the validity of $\underline{2}(\lambda z. z) : \text{Ok}$, but the validity of $\lambda x. x$. For example, we can have a smaller proof where $\underline{2}(\lambda z. z)$ is untouched.

For preservation, although it is true that $Q \triangleright Q'$ and $\Gamma \vdash Q : A, \Delta$ implies $\Gamma \vdash Q' : A, \Delta$ holds unconditionally, the proof is not straightforward. In a typical preservation proof, we take the typing derivation for the term *to be reduced* (Q), and adjust it to a typing derivation for the term *after reduction* (Q'). For a single-succedent system, a derivation that assigns a type to Q is always decided by Q itself. When the term reduces, we can construct a derivation of Q' by using the subderivation that types the (perhaps reduced) subterm of Q . However, in a multi-succedent system, the derivation of $\Gamma \vdash Q : A, \Delta$ might be completely independent from the term Q . Consider the derivation:

$$\begin{array}{c} \text{VAR} \frac{}{x : \text{Int} \vdash \underline{2} : \text{Ok} \rightarrow \text{Ok}, x : \text{Int}} \quad \text{VAR} \frac{}{x : \text{Int} \vdash x : \text{Int}, \underline{3} : \text{Ok}} \\ \text{ABS} \frac{}{\vdash \underline{2} : \text{Ok} \rightarrow \text{Ok}, \lambda x. x : \text{Int} \rightarrow \text{Int}} \quad \text{ABS} \frac{}{\vdash \underline{3} : \text{Ok}, \lambda x. x : \text{Int} \rightarrow \text{Int}} \\ \text{APP} \frac{}{\vdash \underline{2} \underline{3} : \text{Ok}, \lambda x. x : \text{Int} \rightarrow \text{Int}} \end{array}$$

In such cases, reconstructing a derivation for Q' becomes problematic because the subderivations might bear no relation to Q . To address this problem, we need a way to identify which conclusion formulas are *relevant* to the proof.

However, this task is far from straightforward. Vertically, the *principal* (or *focused*) term can shift arbitrarily among conclusions at each inference step, introducing redundancy at varying levels. Horizontally, the relevance can interfere among terms: a typing assignment might only be meaningful when another typing assignment in the same conclusion is meaningful. Moreover, a typing judgement may also have many different derivations, each with different relevant formulas in their conclusion, for example when, in $\vdash M : A, N : B$, both $\vdash M : A$ and $\vdash N : B$ are independently provable. This reflects a deeper issue: the typing system lacks an internal mechanism to track the semantic justification (i.e., the root cause) of a typing judgment.

To bridge the gap between derivations and their semantics, we introduce an additional precondition $\Gamma \not\vdash \Delta$, meaning $\Gamma \vdash \Delta$ is *not derivable*, for the preservation proof. That is, we show $\Gamma \vdash Q : A, \Delta$ implies $\Gamma \vdash Q' : A, \Delta$ whenever $\Gamma \not\vdash \Delta$. This allows us to prove preservation via standard induction on typing rules just as in a single-succedent (traditional) type system, without delving into the complexities of tracking stepwise dependencies across all possible derivations. Finally, we demonstrate that this assumption can be eliminated, restoring the standard preservation theorem without additional premises. This approach provides a purely syntactic method to handle irrelevance in multi-succedent systems, bypassing the need for explicit relevance constraints.

THEOREM 7.4 (PROGRESS AND PRESERVATION). *Suppose all terms in the following are closed:*

Preservation: Suppose $Q \triangleright Q'$. If $\Gamma \vdash Q : A, \Delta$, then $\Gamma \vdash Q' : A, \Delta$.

Progress: If $\vdash \Delta$, then there is some formula $M : A \in \Delta$ such that, if M is a normal form, then:

- $A \sqsubseteq \text{Ok}$ implies M is a value
- and, $A \sqsubseteq \text{Ok}^c$ implies M is stuck.

A corollary of this theorem is that $M : \text{Ok} \vdash$ implies that M either diverges or gets stuck, since $\vdash M : \text{Ok}^c$ follows from this judgement by (COMPR).

Two properties of our system distinguish its progress theorem from conventional formulations. The first is the multi-succedent nature. Unlike preservation, which applies uniformly to all conclusion terms, progress only guarantees that some term in the conclusion satisfies the progress statement. This reflects a reliance on the underlying semantics that justify why the statement

holds. The second is its ability to type normal forms in general. In standard systems, progress rules out non-value normal forms (i.e., stuck terms) by appealing to canonical forms of values, ensuring all non-values are reducible. By contrast, our system does not exclude non-value normal forms. Instead, our progress theorem adopts a different perspective: a normal form of the type Ok^c is guaranteed to go wrong, and a normal form a value type is guaranteed to not go wrong. This motivated the revealing of another property of our system: any normal-form cannot be given both a type and its complement.

THEOREM 7.5 (NORMAL FORM CUT ADMISSIBILITY). *If $\Gamma \vdash U : A, \Delta$ and $\Gamma \vdash U : A^c, \Delta$, then $\Gamma \vdash \Delta$*

From progress and preservation we obtain the soundness of the type system, that is, if $\vdash M : \text{Ok}$ then M cannot go wrong – it either diverges or reduces to a value, and if $\vdash M : \text{Ok}^c$, then M cannot evaluate – it either diverges or goes wrong. As is typical, the system is not *complete*, that is, there exist closed terms M for which neither $\vdash M : \text{Ok}$, nor $\vdash M : \text{Ok}^c$. An example is if $2 < 3$ then $\underline{1}$ else $(\underline{1} \ \underline{1})$, for which deducing a type of Ok would need more precise information about the behaviour of the relation $<$ in the guard than is available in the system.

However, a remarkable fact about our system is that it is complete for M in normal form. Given that the type system is designed to allow us to prove properties A of the reduction behaviour of terms M , expressed as $M : A$, it seems quite basic that we should want that the system is, at the very least, capable of proving properties of terms in normal form, whose reduction behaviour is completely trivial. However, this basic property is not satisfied by most traditional type systems, even for values! A function value whose body happens to be a stuck term, e.g. $\lambda x. \underline{2} \ x$, is not usually typable, since the standard (ABS) rule would require that we can type the stuck term $\underline{2} \ x$.

THEOREM 7.6 (COMPLETE CLASSIFICATION OF NORMAL FORMS). *For all normal forms U , both of the following are true:*

- (i) *If U is a value then $\vdash U : \text{Ok}$.*
- (ii) *If U is stuck then $\vdash U : \text{Ok}^c$*

For values, this is perhaps not so interesting, because we are very familiar with the typing of values. For stuck terms, it is more illuminating, because here it is not so clear *a priori* that we have the “right” set of typing and subtyping rules. Consider a stuck term of shape $\text{match } V \text{ with } \{ |_{i=1}^k p_i \mapsto N_i \}$, which is stuck by virtue of the fact that value V doesn’t match any of the patterns p_i . Completeness requires that we can certify that this term is stuck by a derivation. To do this, say in the one-sided system (which is equivalent to the two-sided system), there is essentially no choice but to use (MATCHE), and hence it must be shown that: from $\forall i. \forall \sigma. V \neq p_i \sigma$ it follows that $\vdash V : (\bigvee_{i=1}^k p_i[\text{Ok}/x \mid x \in \text{FV}(p_i)])^c$. This latter judgement is the single premise of (MatchE). In other words, Theorem 7.6 reveals, amongst other things, that pattern (non-)matching of V , i.e. the condition $\forall i. \forall \sigma. V \neq p_i \sigma$ can be captured precisely by the type system.

8 Conclusion and Related Work

We introduced a new, two-sided type system that overcomes some significant weaknesses of the systems in [28]. The system supports a wide range of refutation principles, including a type-theoretic of the coimplication from bi-intuitionistic logic. These were put to use certifying that a number of Erlang-like programs go wrong. The system is sound and, in a sense, complete for normal forms.

Type inference. We did not consider type inference, but we can observe that, barring the (COMP) and (SUBR) rules, in all of the rules of the one-sided system the terms in the premises are strictly smaller than those in the conclusion. Hence, the system naturally suggests a standard approach in which subsumption is first absorbed into the other rules, and then a notion of canonical derivation(s) defined, in which fresh type variables are introduced at synthesis points and constrained according

to subtyping. However, since the rules are not syntax directed – one may freely choose between typings on the right – there is some challenge to make such a scheme work efficiently.

8.1 Related Work

Two-sided type systems. Our work is based on two-sided type systems introduced in [28]. Compared to the systems in that paper, our system overcomes the three weaknesses listed in Remarks 3.6, 4.3 and 7.3. Thanks to the notion of relevance and the cut theorem, we were able to prove soundness for all judgements, and not only those that satisfy the single-subject constraint. Our system contains both (COMPL) and (COMPR) and yet still enjoys both *well-typed programs cannot go wrong* and *ill-typed programs cannot evaluate*. Our system is made from a small number of well-motivated rules, with the key left rules of [28] all derivable. Moreover, our (MATCH) rule exploits the two-sidedness of the system and the general soundness result. However, we don't consider type inference.

By encoding atoms as unary constructors, the constrained type system of [28, Section 6] can handle the first two of Héberts examples, but not the third: when typing an expression of shape $\text{match } x \text{ with } \{ |_{i=1}^k p_i \mapsto M_i \}$, a type A must be given that can be refuted for both of the branches simultaneously, since there is no capability to refute that a variable x in the scrutinee can match any particular pattern. Their system could be extended with such a capability by adding e.g. a notion of type disjointness in the subtype system. This way, assuming a type $x : A$ would be sufficient to refute an assignment $x : B \vdash$ whenever A and B disjoint, which could be used to ignore a case in the match and thus derive a typing expressing the dependency of just a single branch. Currently, the system of [28, Section 6] only allows reasoning about type disjointness for concrete values, but not for variables. With this extension, their system would be able to verify the third example.

Typing for incorrectness. One half of our system, and the part we have concentrated on in this paper, has as its purpose to reason about incorrectness by refuting type assignments. Probably the best known type system for reasoning about incorrectness is Lindahl and Sagonas' *success types*, which have been popularised in the Erlang community through the *Dialyzer* tool [17]. As argued by Sagonas in his Erlang'21 keynote, the *Dialyzer* tool has been very successful [30]. It is part of the standard Erlang platform OTP [12], and is widely used by the community. Yet, outside of this community (and those of Erlang's sibling languages), type systems for finding bugs, that is for reasoning about incorrectness, are largely unknown. One may wonder why the single most popular developer tool for Erlang programmers [20] has not been replicated elsewhere, for example through success typing for other dynamically typed programming languages? Or why there is no parallel to the substantial and varied research that we see into new and more powerful type systems for correctness? One reason could be that there lacks a unified foundation for typing for (in)correctness, and we hope that the present work can contribute there.

Alternative type systems for incorrectness, include work by Jakob and Thiemann [14], who attempt to characterise success types as a kind of refutation in a system in which one constructs a logical formula to describe inputs that guarantee to crash a function, and the *coverage types* of [34], which embed "must-style", underapproximate reasoning designed for checking the coverage of generators in property-based testing.

There are also works in which types are used as a language for properties, but the verification that a program satisfies such a property is based on the semantics directly, rather than via a proof (type) system. For example, Carrott et al. [2] and Qian et al. [26], both concern type safety bugs. However, rather than deriving a proof in a type system, they instead reduce the problem to deciding a reachability problem concerning a transformation of the program. Both works show the effectiveness of solving their reachability problem by symbolic execution.

One application of our system is to provide more precise type errors. Webbers et al. [33] improve the type error messages from refinement type checking and inference. For each non-provable judgment, an explicit “refutation” is produced to explain why the judgement is not provable, but it does not necessarily imply something about the behaviour of the subject (e.g. that it will crash).

Logics for incorrectness. Incorrectness logic of O’Hearn [21] has sparked a new interest in systems for reasoning about programs that go wrong. Subsequent work has extended its scope to languages with concurrency [27], demonstrated its real-world effectiveness [15] and spawned alternatives such as the *Outcome Logic* of Zilberstein et al. [35], which additionally supports reasoning about correctness. A key feature of these logics is *under-approximation* as a means to achieve true positives. In incorrectness logic, states in a postcondition must be *reachable*. This leads to the need to reason about termination, but it also allows for an analysis to dynamically drop disjunctions in order to scale well. The foundation of our system is not under-approximation, but necessity (via complement) – when $M : A$ is provable, it remains the case that the type A is an *over-approximation* of M . This makes the system closer to the works of Cousot et al. [7, 8] and Mackay et al. [18]. The former considers the problem of inferring *necessary preconditions* in order to detect contract violations that will lead to assertion failures. The latter uses necessity as a means to specify the *robustness* of module specifications in object-oriented programming.

Type systems with complement. A small number of systems in the literature are equipped with a complement, or negation type operator. However, to the best of our knowledge, none allow for reasoning about incorrectness by proving that programs go wrong. A complement operator appears in the system of Aiken et al. [1] and a negation in the work of Parreaux and Chau [22]. Both systems are more sophisticated than ours, with the latter giving a treatment of a realistic programming language and a detailed and insightful account of type inference – in fact this is the main motivation for the complement in both of these papers. A complement operator is also present in the line of work on set-theoretic types and semantic subtyping (see e.g. [3–5]) in which the subtyping relation is defined directly as inclusion of the induced sets. A much simpler system is that of [23], which lacks function types, and axiomatises a subtyping relation that is shown complete with respect to a set theoretic model. Finally, in the dependent type system of Unno et al. [32], no typable program can go wrong, but the system is so expressive that one can compute directly the complements of arbitrary types without the need for a formal operator.

Coimplication and bi-intuitionistic logic. Bi-intuitionistic logic extends intuitionistic logic with a coimplication (also called subtraction, or pseudo-difference), dual to implication. It was first studied by Rauszer [29], but the cut-elimination result was later shown to be flawed by Pinto and Uustalu [24]. A version of the Curry-Howard correspondence has been given for a version of bi-intuitionistic logic by Crolard [10], but it is for a natural deduction system, in the style of Parigot’s $\lambda\mu$ -calculus. Coimplication is at the heart of the work of Choudhury and Gay [6], though in the context of language design. Whereas our paper views a type as a property, they are interested in designing a calculus corresponding to the logical content of the type. In particular, rather than viewing coimplication as a property of functions, they design a calculus in which the proofs of coimplications have their own term construction, the coabstraction.

Acknowledgments

We are grateful for many improvements suggested by the anonymous reviewers, and for useful discussions with our colleagues in the Programming Languages Research Group.

References

- [1] Alexander Aiken, Edward L. Wimmers, and T. K. Lakshman. 1994. Soft Typing with Conditional Types. In *Conference Record of POPL '94: 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Portland, Oregon, USA, January 17-21, 1994*. 163–173. <https://doi.org/10.1145/174675.177847>
- [2] Pedro Carrott, Sacha-Élie Ayoun, and Azalea Raad. 2025. Compositional Bug Detection for Internally Unsafe Libraries: A Logical Approach to Type Unsoundness. In *39th European Conference on Object-Oriented Programming (ECOOP 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 333)*, Jonathan Aldrich and Alexandra Silva (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 5:1–5:28. <https://doi.org/10.4230/LIPIcs.ECOOP.2025.5>
- [3] Giuseppe Castagna. 2024. *Programming with Union, Intersection, and Negation Types*. Springer International Publishing, Cham, 309–378. https://doi.org/10.1007/978-3-031-34518-0_12
- [4] Giuseppe Castagna, Mickaël Laurent, Kim Nguyễn, and Matthew Lutze. 2022. On Type-Cases, Union Elimination, and Occurrence Typing. *Proc. ACM Program. Lang.* 6, POPL, Article 13 (Jan. 2022), 31 pages. <https://doi.org/10.1145/3498674>
- [5] Giuseppe Castagna and Zhiwu Xu. 2011. Set-theoretic foundation of parametric polymorphism and subtyping. In *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming (Tokyo, Japan) (ICFP '11)*. Association for Computing Machinery, New York, NY, USA, 94–106. <https://doi.org/10.1145/2034773.2034788>
- [6] Vikraman Choudhury and Simon J. Gay. 2025. The Duality of λ -Abstraction. *Proc. ACM Program. Lang.* 9, POPL, Article 12 (Jan. 2025), 30 pages. <https://doi.org/10.1145/3704848>
- [7] Patrick Cousot, Radhia Cousot, Manuel Fähndrich, and Francesco Logozzo. 2013. Automatic Inference of Necessary Preconditions. In *Verification, Model Checking, and Abstract Interpretation*, Roberto Giacobazzi, Josh Berdine, and Isabella Mastroeni (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 128–148. https://doi.org/10.1007/978-3-642-35873-9_10
- [8] Patrick Cousot, Radhia Cousot, and Francesco Logozzo. 2011. Precondition Inference from Intermittent Assertions and Application to Contracts on Collections. In *Verification, Model Checking, and Abstract Interpretation*, Ranjit Jhala and David Schmidt (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 150–168. https://doi.org/10.1007/978-3-642-18275-4_12
- [9] Tristan Crolard. 2001. Subtractive logic. *Theoretical Computer Science* 254, 1 (2001), 151–185. [https://doi.org/10.1016/S0304-3975\(99\)00124-3](https://doi.org/10.1016/S0304-3975(99)00124-3)
- [10] Tristan Crolard. 2004. A Formulae-as-Types Interpretation of Subtractive Logic. *J. Log. Comput.* 14, 4 (2004), 529–570. <https://doi.org/10.1093/LOGCOM/14.4.529>
- [11] Jana Dunfield and Neel Krishnaswami. 2021. Bidirectional Typing. *ACM Comput. Surv.* 54, 5, Article 98 (May 2021), 38 pages. <https://doi.org/10.1145/3450952>
- [12] Erlang/OTP 28. 2025. Dialyzer v5.4. Last accessed: 1/7/2025.
- [13] Fred Hébert. 2013. *Learn You Some Erlang for Great Good!* No Starch Press. As of 6/6/2025, the examples can be viewed directly at <https://learnyousomeerlang.com/dialyzer#type-inference-and-discrepancies>.
- [14] Robert Jakob and Peter Thiemann. 2015. A Falsification View of Success Typing. In *NASA Formal Methods*, Klaus Havelund, Gerard Holzmann, and Rajeev Joshi (Eds.). Springer International Publishing, Cham, 234–247. https://doi.org/10.1007/978-3-319-17524-9_17
- [15] Quang Loc Le, Azalea Raad, Jules Villard, Josh Berdine, Derek Dreyer, and Peter W. O'Hearn. 2022. Finding Real Bugs in Big Programs with Incorrectness Logic. *Proc. ACM Program. Lang.* 6, OOPSLA1, Article 81 (apr 2022), 27 pages. <https://doi.org/10.1145/3527325>
- [16] Celia Mengyue Li, Sophie Pull, and Steven Ramsay. 2025. A Complementary Approach to Incorrectness Typing. <https://doi.org/10.48550/arXiv.2510.13725> arXiv:2510.13725 [cs.PL]
- [17] Tobias Lindahl and Konstantinos Sagonas. 2006. Practical Type Inference Based on Success Typings. In *Proceedings of the 8th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming (Venice, Italy) (PPDP '06)*. Association for Computing Machinery, New York, NY, USA, 167–178. <https://doi.org/10.1145/1140335.1140356>
- [18] Julian Mackay, Susan Eisenbach, James Noble, and Sophia Drossopoulou. 2022. Necessity Specifications for Robustness. *Proc. ACM Program. Lang.* 6, OOPSLA2, Article 154 (oct 2022), 30 pages. <https://doi.org/10.1145/3563317>
- [19] Robin Milner. 1978. A theory of type polymorphism in programming. *J. Comput. System Sci.* 17, 3 (1978), 348–375. [https://doi.org/10.1016/0022-0000\(78\)90014-4](https://doi.org/10.1016/0022-0000(78)90014-4)
- [20] Tamás Nagy and Anikó Nagyné Víg. 2008. Erlang testing and tools survey. In *Proceedings of the 7th ACM SIGPLAN Workshop on ERLANG (Victoria, BC, Canada) (ERLANG '08)*. Association for Computing Machinery, New York, NY, USA, 21–28. <https://doi.org/10.1145/1411273.1411277>
- [21] Peter W. O'Hearn. 2019. Incorrectness Logic. *Proc. ACM Program. Lang.* 4, POPL, Article 10 (dec 2019), 32 pages. <https://doi.org/10.1145/3371078>
- [22] Lionel Parreaux and Chun Yin Chau. 2022. MLstruct: Principal Type Inference in a Boolean Algebra of Structural Types. *Proc. ACM Program. Lang.* 6, OOPSLA2, Article 141 (oct 2022), 30 pages. <https://doi.org/10.1145/3563304>

- [23] David J. Pearce. 2013. Sound and Complete Flow Typing with Unions, Intersections and Negations. In *Verification, Model Checking, and Abstract Interpretation*, Roberto Giacobazzi, Josh Berdine, and Isabella Mastroeni (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 335–354.
- [24] Luis Pinto and Tarmo Uustalu. 2009. Proof Search and Counter-Model Construction for Bi-intuitionistic Propositional Logic with Labelled Sequents. In *Automated Reasoning with Analytic Tableaux and Related Methods*, Martin Giese and Arild Waaler (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 295–309.
- [25] Andrew M. Pitts. 2012. Lecture Notes on Types for Part II of the Computer Science Tripos. (2012). Page 2. Available from <https://www.cl.cam.ac.uk/teaching/1213/Types/typ-notes.pdf>, last accessed 18/07/2025.
- [26] Kelvin Qian, Scott Smith, Brandon Stride, Shiwei Weng, and Ke Wu. 2024. Semantic-Type-Guided Bug Finding. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 348 (Oct. 2024), 28 pages. <https://doi.org/10.1145/3689788>
- [27] Azalea Raad, Josh Berdine, Derek Dreyer, and Peter W. O’Hearn. 2022. Concurrent Incorrectness Separation Logic. *Proc. ACM Program. Lang.* 6, POPL, Article 34 (jan 2022), 29 pages. <https://doi.org/10.1145/3498695>
- [28] Steven Ramsay and Charlie Walpole. 2024. Ill-Typed Programs Don’t Evaluate. *Proc. ACM Program. Lang.* 8, POPL, Article 67 (Jan 2024), 31 pages. <https://doi.org/10.1145/3632909>
- [29] Cecylia Rauszer. 1974. A Formalization of the Propositional Calculus of H-B Logic. *Studia Logica: An International Journal for Symbolic Logic* 33, 1 (1974), 23–34. <http://www.jstor.org/stable/20014691>
- [30] Konstantinos Sagonas. 2021. Fifteen years of successfully dialyzing Erlang and Elixir code (invited talk). In *Proceedings of the 20th ACM SIGPLAN International Workshop on Erlang* (Virtual, Republic of Korea) (*Erlang 2021*). Association for Computing Machinery, New York, NY, USA, 1. <https://doi.org/10.1145/3471871.3480952>
- [31] Luca Tranchini. 2017. Natural deduction for bi-intuitionistic logic. *Journal of Applied Logic* 25 (2017), S72–S96. <https://doi.org/10.1016/j.jal.2017.12.001>
- [32] Hiroshi Unno, Yuki Satake, and Tachio Terauchi. 2017. Relatively Complete Refinement Type System for Verification of Higher-Order Non-Deterministic Programs. *Proc. ACM Program. Lang.* 2, POPL, Article 12 (dec 2017), 29 pages. <https://doi.org/10.1145/3158100>
- [33] Robin Webbers, Klaus von Gleissenthall, and Ranjit Jhala. 2024. Refinement Type Refutations. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 305 (Oct. 2024), 26 pages. <https://doi.org/10.1145/3689745>
- [34] Zhe Zhou, Ashish Mishra, Benjamin Delaware, and Suresh Jagannathan. 2023. Covering All the Bases: Type-Based Verification of Test Input Generators. *Proc. ACM Program. Lang.* 7, PLDI, Article 157 (June 2023), 24 pages. <https://doi.org/10.1145/3591271>
- [35] Noam Zilberstein, Derek Dreyer, and Alexandra Silva. 2023. Outcome Logic: A Unifying Foundation for Correctness and Incorrectness Reasoning. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 93 (apr 2023), 29 pages. <https://doi.org/10.1145/3586045>

Received 2025-07-10; accepted 2025-11-06